# BLUE TEAM / SOC
# Wazuh SIEM Laborprojekt

Dokumentsprache: Deutsch

# Inhaltsverzeichnis

Übersicht der Hauptabschnitte


| Abschnitt | Seite | Abschnitt | Seite |
|---|---|---|---|
| 1. Projektprofil | – | 11. Ergebnisse, Grenzen und Lessons Learned | – |
| 2. Architektur und Datenfluss | – | 12. Portfolio-Struktur für Bewerbungen | – |
| 3. Aufbau der Monitoring-Plattform | – | 13. Entwicklungsroadmap | – |
| 4. Detection Use Cases | – | 14. Nachweisregister | – |
| 5. Endpoint Monitoring | – | 15. Detection Engineering mit Sysmon, Sigma und Suricata | – |
| 6. Active Directory Monitoring | – | 16. SOC-Fallstudie Suricata TCP-SYN-Portscan | – |
| 7. Detection Engineering: Network Scan | – | 17. SOC-Fallstudie Encoded PowerShell | – |
| 8. Ubuntu Hardening und Risk Acceptance | – | 18. SOC-Fallstudie Konten und privilegierte Gruppen | – |
| 9. Sysmon und Encoded PowerShell | – | 19. Dashboard-Übersichten und Darstellung | – |
| 10. Alert Triage und Incident Response | – | 20. MITRE ATT&CK und Ereignistypen | – |

# 1. Projektprofil

## Was aufgebaut wurde und welchen praktischen Nutzen das Projekt zeigt

### Kurzbeschreibung

In einer abgegrenzten Hyper-V-Laborumgebung wurde ein zentraler Blue-Team-Stack mit Wazuh aufgebaut. Windows Server 2022, Windows 11 und Ubuntu liefern sicherheitsrelevante Ereignisse an den Wazuh Manager auf CentOS. Kali Linux wird ausschließlich als autorisiertes Testsystem verwendet. Das Gesamtprojekt verbindet hostbasierte Windows- und Linux-Telemetrie, Sysmon, Sigma-Regeln und Suricata-Netzwerkdaten mit reproduzierbaren Detection-, Triage- und Incident-Response-Abläufen.

### Ergebnisse auf einen Blick

| Ergebnis | Nachweis |
|---|---|
| Zentrale Sicht auf Windows- und Linux-Ereignisse | Drei aktive Agents; Suche und Analyse im Wazuh Dashboard |
| Plattformübergreifendes FIM | Datei added, modified und deleted; Inhaltsänderung über syscheck.diff |
| Windows- und AD-Monitoring | Event IDs 4625, 4720, 4728, 4729 und 4726 |
| Eigene Scan-Erkennung | Regeln 100110 und 100111; korrelierter Level-10-Alert; MITRE T1046 |
| Sysmon/PowerShell Detection | Sysmon Event ID 1; Wazuh-Regel 92057; Level 12; MITRE T1059.001 |
| SCA und Risk Acceptance | Ubuntu-Härtung mit Before/After-Nachweis und begründeter Ausnahme |
| Incident-orientierte Bearbeitung | Triage, Klassifizierung, Runbooks, Bereinigung und Abschlusskontrollen |
| Sigma-Regelentwicklung | whoami /priv; Sigma CLI mit 0 Syntax- und Condition-Fehlern |
| Suricata-Netzwerkerkennung | ICMP und TCP-SYN; Rule 100121, Level 8, MITRE T1046 |

### Nachgewiesene Kompetenzen

- SIEM-Grundbetrieb: Agents, Manager, Indexer, Dashboard und Logquellen prüfen.
- Detection Engineering: Detection-Gap analysieren, Regeln entwickeln, Korrelation testen und MITRE-Techniken zuordnen.
- Alert Triage: Agent, Benutzer, Quelle, Zeit, Rule-ID, Level und Folgeaktivität gemeinsam bewerten.
- Windows Security Monitoring: Active Directory, PowerShell, Sysmon und Windows Event Logs auswerten.
- Linux Security Monitoring: PAM, FIM und CIS-basierte SCA-Ergebnisse technisch bewerten.
- Dokumentation: Ergebnisse, Einschränkungen, Bereinigung und Risk Acceptance nachvollziehbar festhalten.

# 2. Architektur und Datenfluss

Kleine, klar abgegrenzte Laborumgebung mit reproduzierbaren Systemrollen

### Systeme und Rollen

| System | IP-Adresse | Rolle | Wazuh |
|---|---|---|---|
| Windows Host | 192.168.20.1 | Hyper-V, Gateway, NAT | Kein Agent |
| DC-Test | 192.168.20.2 | Windows Server 2022, AD DS, DNS, IIS | Agent 001 |
| ham1 | 192.168.20.5 | Windows 11 Domänenclient, Sysmon | Agent 002 |
| ubuntu-Desktop | 192.168.20.20 | Ubuntu Desktop, FIM, PAM, SCA | Agent 003 |
| Kali Linux | 192.168.20.7 | Autorisierte Tests | Kein Agent |
| Wazuh Server | 192.168.20.8 | Manager, Indexer, Dashboard | Lokaler Server |

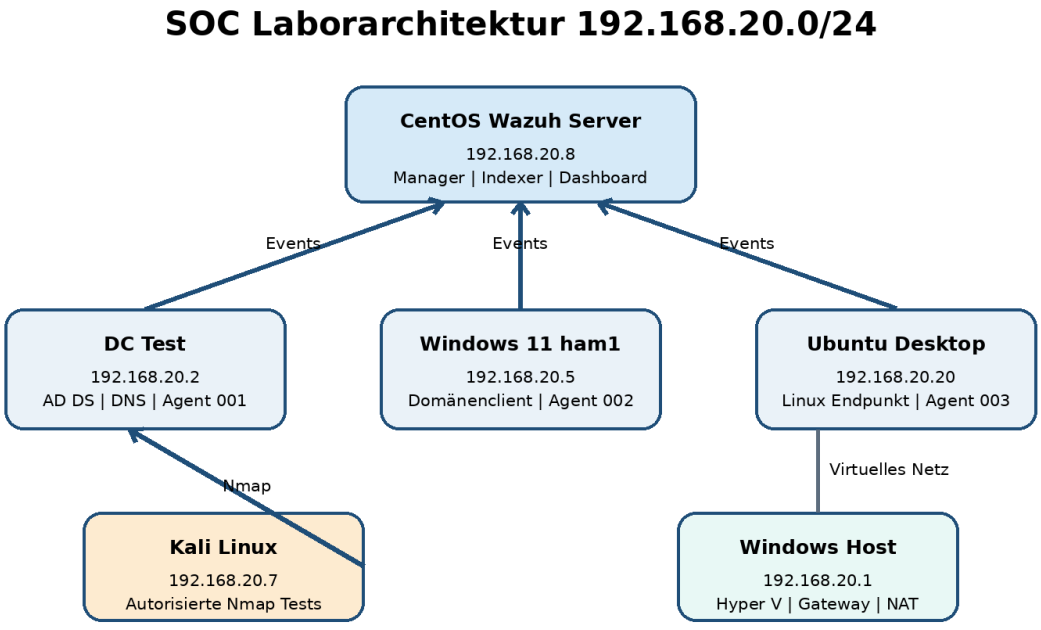



*Abbildung 1: Architektur und Datenfluss der SOC-Laborumgebung*

### Datenfluss

Die Endpunkte senden ihre Ereignisse an den Wazuh Manager. Dort werden die Daten dekodiert, mit Standard- und lokalen Regeln bewertet und im Indexer gespeichert. Das Dashboard stellt Alerts für Suche, Filterung und Triage bereit. Feste IP-Adressen und konsistente Zeitstempel erleichtern die Zuordnung und Korrelation.

**Laborgrenze** Tests ausschließlich im internen Labornetz; keine externen Systeme.

# 3. Aufbau der Monitoring-Plattform

## Vom Grundbetrieb bis zur überprüften Ereigniskette

### Wazuh-Komponenten

Auf CentOS laufen Wazuh Manager, Indexer und Dashboard. DC-Test, ham1 und ubuntu-Desktop wurden als Agents registriert. Die Datenerfassung wurde auf relevante Quellen begrenzt, um die Analyse übersichtlich zu halten.

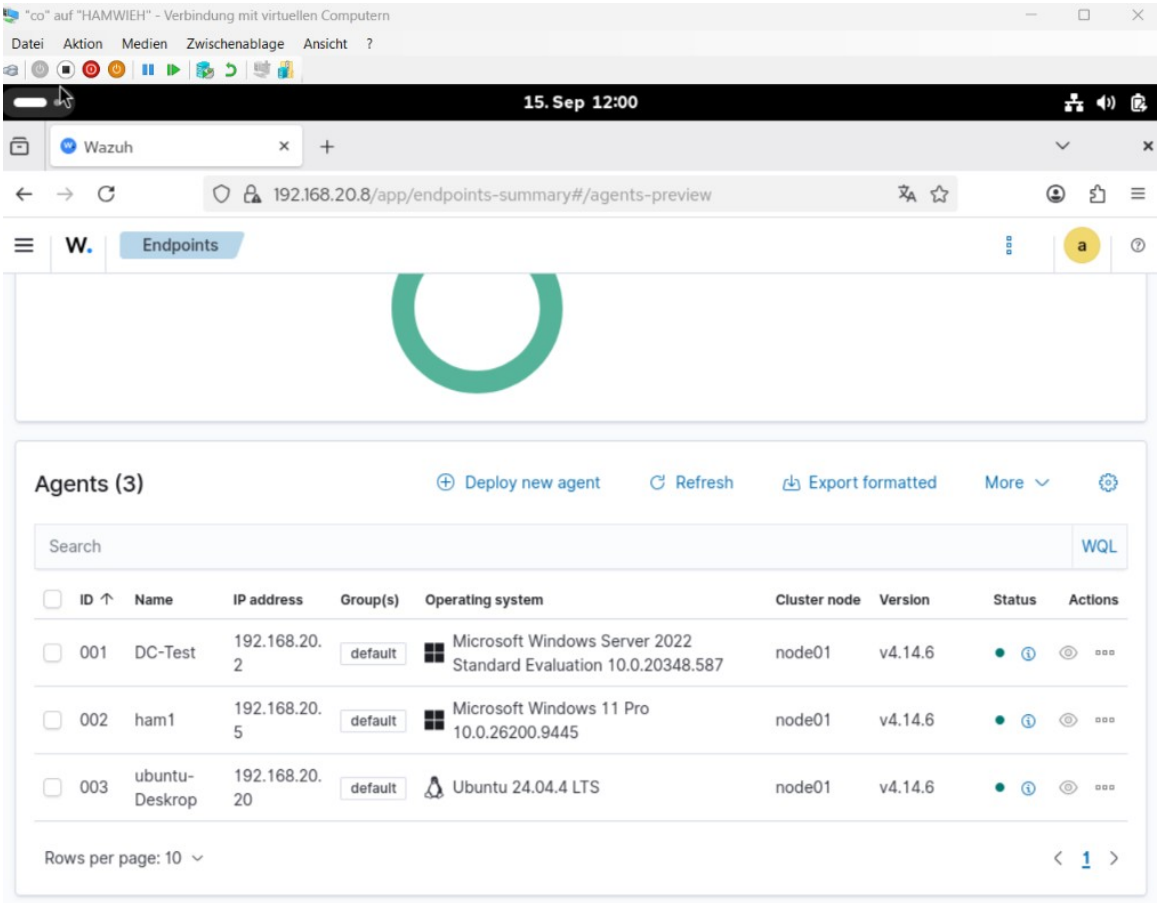



*Abbildung 2: Drei aktive Wazuh Agents mit den zugeordneten Endpunkten*

### Qualitätssicherung

- Jeder Use Case wurde kontrolliert ausgelöst und zuerst lokal, danach im SIEM geprüft.
- Zu jedem erfolgreichen Alert wurden mindestens Agent, Event- oder Rule-ID, Level und Kontext kontrolliert.
- Eigene Regeln wurden vor dem Neustart syntaktisch validiert.
- Temporäre Konten und privilegierte Änderungen wurden nach dem Test entfernt und erneut geprüft.
- Fehlende Alerts wurden getrennt nach Auditierung, Transport, Decoding und Regelwerk untersucht.

# 4. Detection Use Cases

## Getestete Ereignisse, Regeln und Analystenbewertung

| Use Case | Datenquelle | Erkennung | Bewertung |
|---|---|---|---|
| Ubuntu FIM | Wazuh FIM | Rules 554, 550, 553; syscheck.diff | Erfüllt |
| Ubuntu Authentifizierung | PAM / auth logs | Rule 5503, Level 5 | Erfüllt |
| Windows Anmeldung | Security Log | Event 4625 / Rule 60122, Level 5 | Erfüllt |
| PowerShell Script Block | PowerShell Operational | Event 4104 | Erfüllt |
| AD Kontolebenszyklus | Security Log | 4720, 4728, 4729, 4726 | Erfüllt |
| Privilegierte Gruppe | Security Log | Rule 60159 | Erfüllt |
| Network Scan | Windows Filtering Platform | 100110 + 100111, Level 10, T1046 | Erfüllt |
| Ubuntu SCA | Wazuh SCA | Check 35517 Passed; 35509 dokumentierte Ausnahme | Bewertet |
| Sysmon PowerShell | Sysmon Event ID 1 | Rule 92057, Level 12, T1059.001 | Erfüllt |
| Sysmon Privilege Discovery | Sysmon Event ID 1 | Custom Rule 100120; whoami /priv | Erfüllt |
| Suricata ICMP | eve.json / SID 1000001 | Wazuh Rule 86601 | Erfüllt |
| Suricata TCP SYN Scan | eve.json / SID 1000002 | Rule 100121, Level 8, T1046 | Erfüllt |

### Bewertungsprinzip

Ein Event ist nur ein technischer Datensatz. Ein Alert zeigt, dass eine Regel ausgelöst wurde. Erst die Triage entscheidet, ob daraus ein Incident entsteht. Deshalb wurde nie nur die Alertstufe betrachtet, sondern immer auch Quelle, Benutzer, Zeitpunkt, Autorisierung und Folgeaktivität.

# 5. Endpoint Monitoring

## Dateiänderungen, Anmeldungen und PowerShell-Aktivität

### File Integrity Monitoring

FIM wurde auf Ubuntu und Windows 11 für ausgewählte Pfade eingerichtet. In kontrollierten Tests wurden Dateien erstellt, geändert und gelöscht. Wazuh zeigte die Aktionen added, modified und deleted; bei Änderungen war zusätzlich der Inhalt über syscheck.diff nachvollziehbar. Die Pfade wurden bewusst eng gewählt, damit kein unnötiges Eventvolumen entsteht.

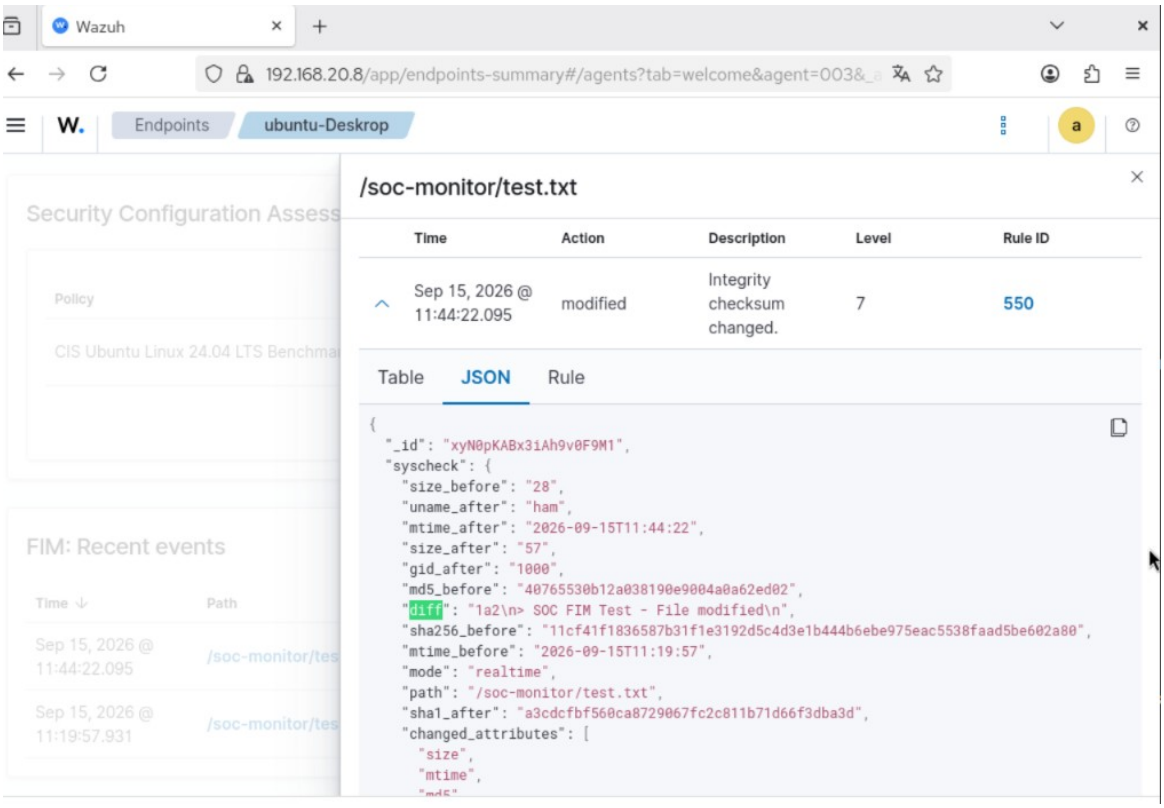



*Abbildung 3: FIM-Detailansicht mit dokumentierter Inhaltsänderung auf Ubuntu*

### Fehlgeschlagene Windows-Anmeldung

Auf ham1 wurde ein fehlgeschlagener Anmeldeversuch erzeugt. Windows protokollierte Event ID 4625. Wazuh übernahm den Datensatz als Rule 60122 auf Level 5. Für die Triage sind Benutzer, Quelladresse, Anmeldetyp, Häufigkeit und erfolgreiche Folgeanmeldungen relevant.

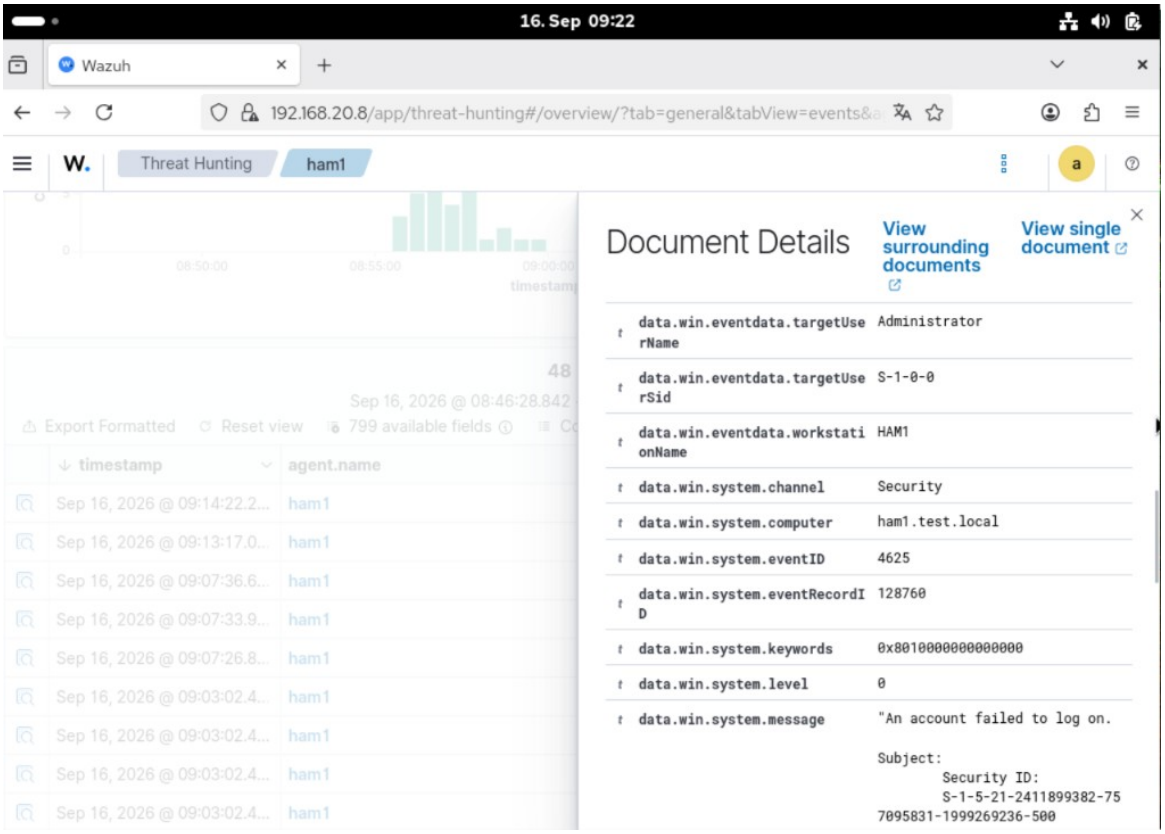



*Abbildung 4: Wazuh-Nachweis des Windows-Ereignisses 4625 auf ham1*

## PowerShell Script Block Logging

Ein kontrollierter Test erzeugte Event ID 4104. PowerShell ist zugleich Administrationswerkzeug und möglicher Angriffsweg; deshalb reicht die Event-ID allein nicht für eine Incident-Entscheidung. Inhalt, Benutzerkontext, Elternprozess und weitere Ereignisse müssen zusammen bewertet werden.

# 6. Active Directory Monitoring

## Privilegierte Änderungen erkennen und vollständig bereinigen

Auf DC-Test wurde ein temporäres Konto angelegt, zu Domain Admins hinzugefügt, wieder entfernt und danach gelöscht. Die Ereignisse 4720, 4728, 4729 und 4726 bildeten den Lebenszyklus ab. Die privilegierte Gruppenänderung wurde zusätzlich über Rule 60159 hervorgehoben.

| Schritt | Windows Event | Analystenfrage |
|---|---|---|
| Konto angelegt | 4720 | Wer hat das Konto wann erstellt? |
| Zu Domain Admins hinzugefügt | 4728 | Gab es eine genehmigte Änderung? |
| Aus Domain Admins entfernt | 4729 | Ist die privilegierte Mitgliedschaft vollständig beendet? |
| Konto gelöscht | 4726 | Bleiben Gruppenmitgliedschaften oder weitere Artefakte zurück? |

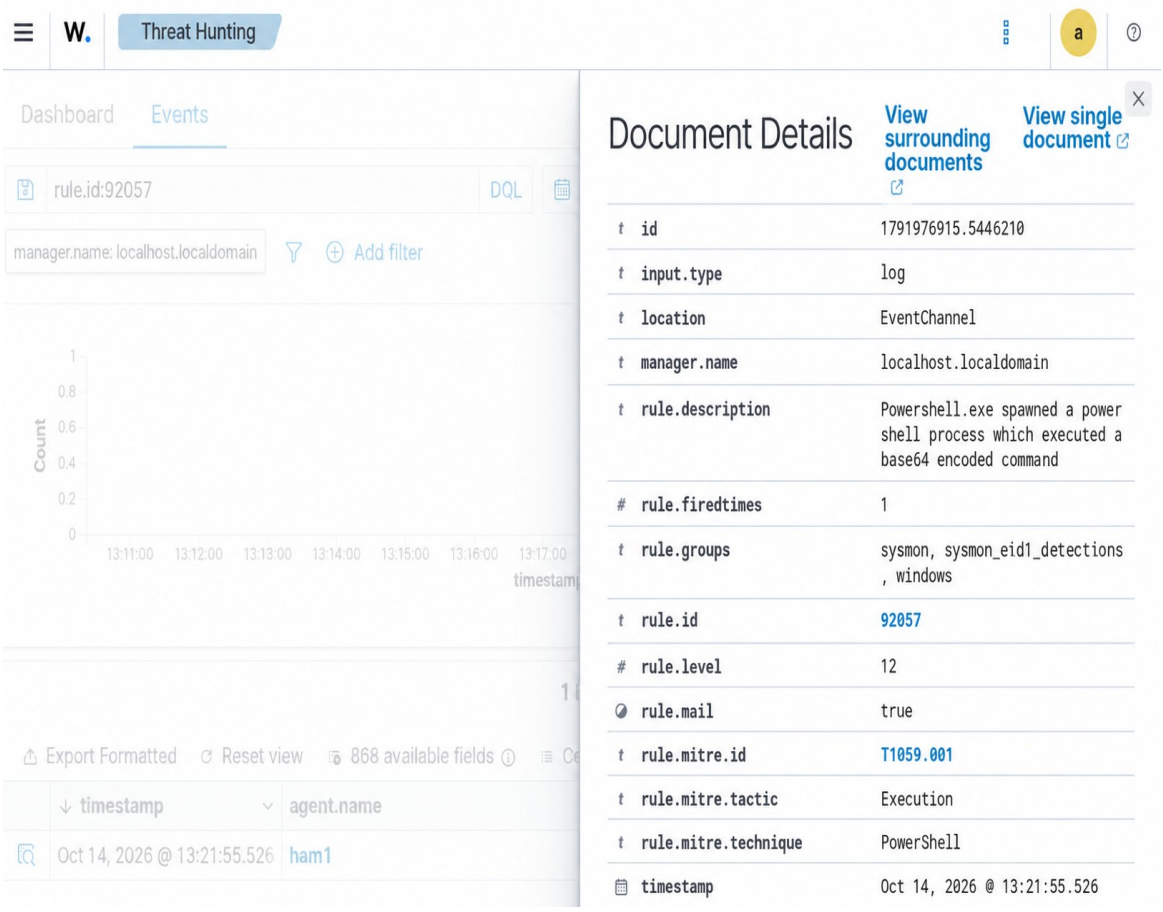



*Abbildung 5: Wazuh-Regel 60159 für eine Änderung der Domain Admins*

### Abschlusskontrolle

Nach dem Test wurde überprüft, dass das temporäre Konto nicht mehr existiert und keine Mitgliedschaft in Domain Admins verbleibt. Damit wurde nicht nur die Erkennung, sondern auch die sichere Bereinigung nachgewiesen.

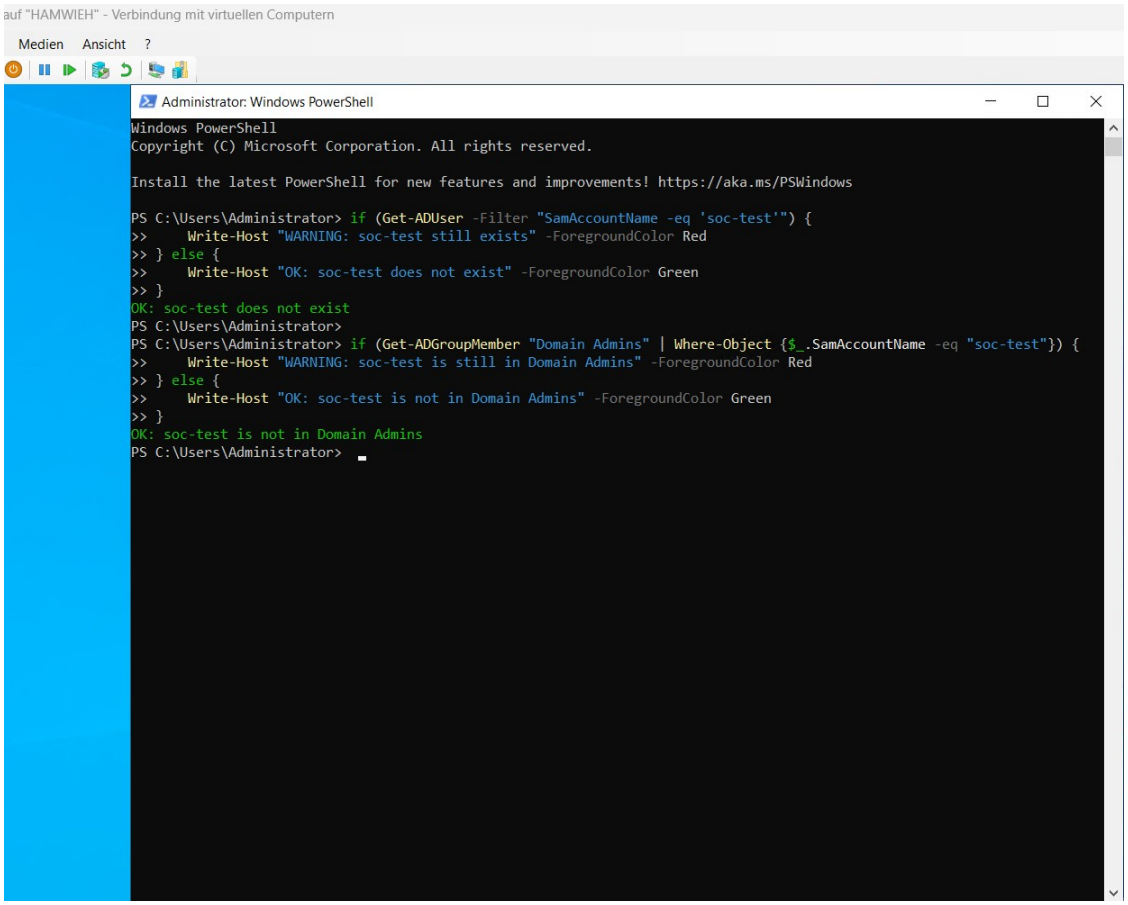



*Abbildung 6: Abschlusskontrolle des entfernten AD-Testkontos*

# 7. Detection Engineering: Network Scan

## Vom Rohereignis zur korrelierten Detection

### Ausgangslage und Detection-Gap

Ein begrenzter Nmap-Test von Kali Linux gegen DC-Test erzeugte Windows-Filtering-Platform-Ereignisse mit Event ID 5156. Die Netzwerkaktivität war im Betriebssystem sichtbar, führte aber zunächst nicht zu einem passenden Wazuh-Alert. Der Agent funktionierte; es fehlte eine projektspezifische Regel.

### Regellogik

| Regel | Funktion | Ergebnis |
|---|---|---|
| 100110 | Erkennt passende 5156-Ereignisse von der bekannten Kali-Quelle 192.168.20.7. | Einzelereignis, Level 5 |
| 100111 | Korreliert mehrere Treffer der Basisregel innerhalb des definierten Zeitfensters. | Scan-Alert, Level 10 |
| MITRE T1046 | Ordnet das Verhalten Network Service Discovery zu. | Analystenkontext |

Die Regeln wurden mit wazuh-analysisd -t validiert; der Test endete mit Exit Code 0. Der Alert war ein True Positive, aber wegen des genehmigten Laborversuchs kein Incident.

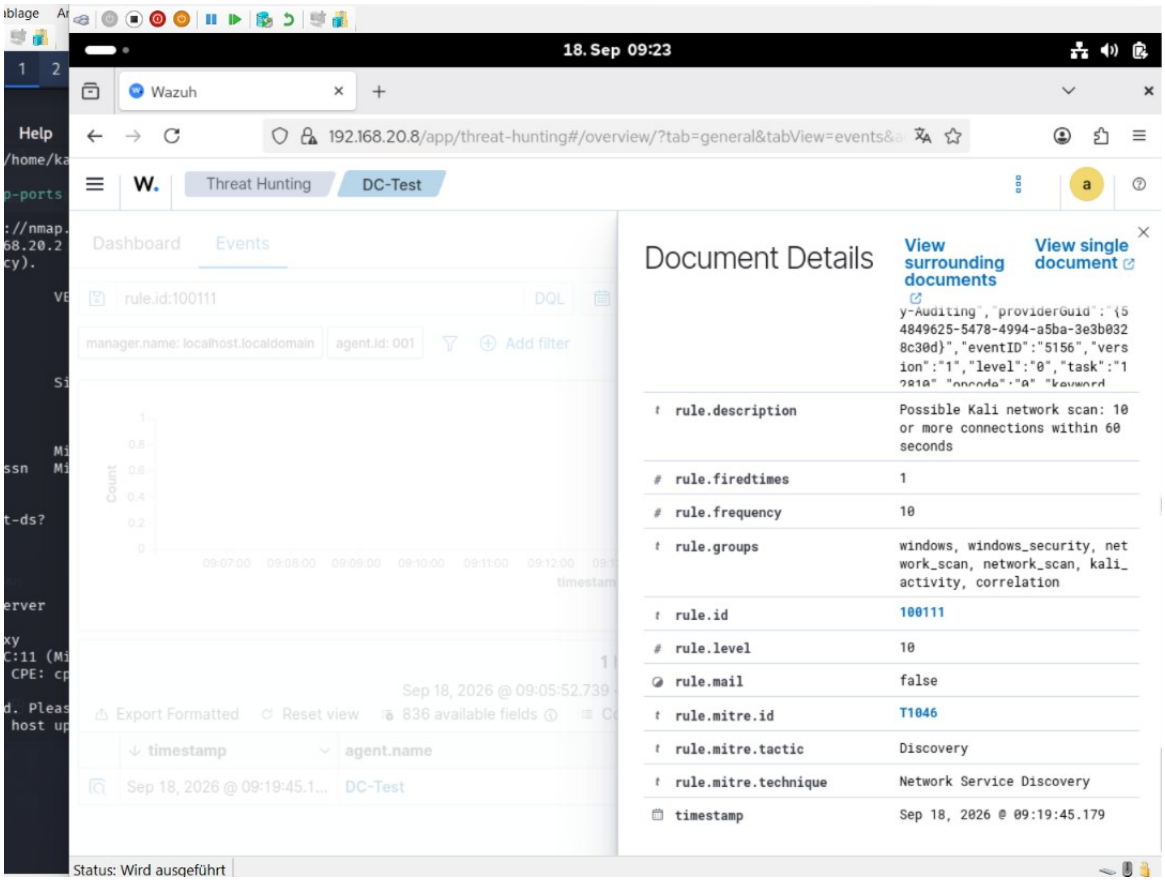



*Abbildung 7: Korrelierter Nmap-Alert der Regel 100111 auf Level 10*

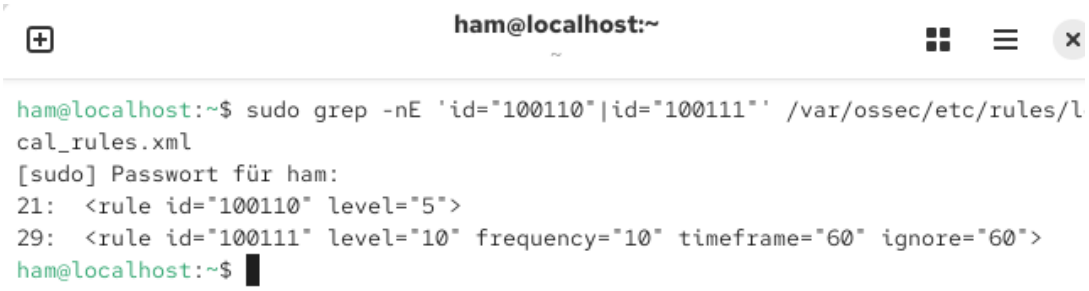



*Abbildung 8: Erfolgreiche Validierung der lokalen Regeln*

**Produktionshinweis** Die feste Kali-IP eignet sich für den reproduzierbaren Labortest, aber nicht als allgemeine produktive Scan-Erkennung. Eine produktive Regel benötigt verhaltensbasierte Kriterien, Ausnahmen und False-Positive-Tests.

# 8. Ubuntu Hardening und Risk Acceptance

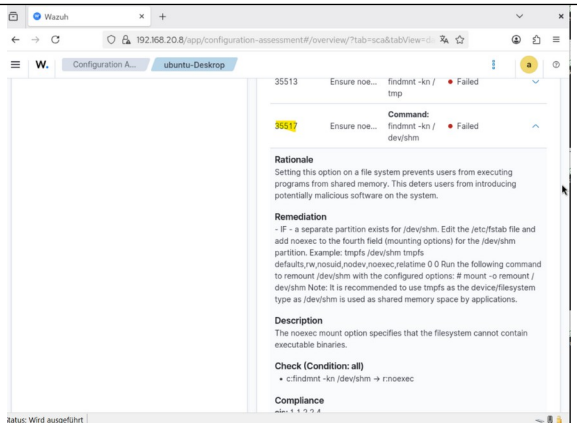
## Sicherheitswert verbessern, ohne Systemfunktion zu verfälschen

### SCA-Auswertung

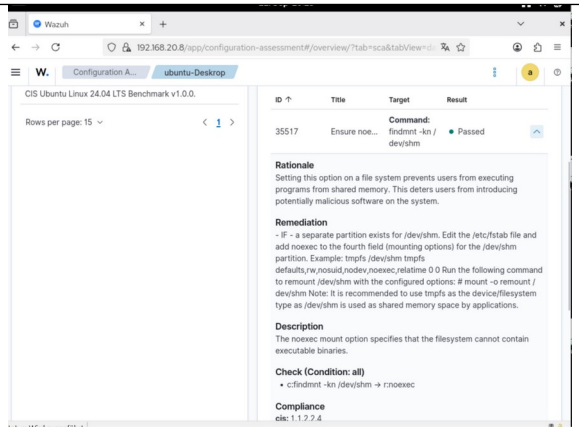
Die CIS-basierte Wazuh-SCA-Policy wurde auf Ubuntu ausgewertet. Der Ausgangswert lag bei 50 Prozent. Zwei Prüfungen wurden technisch vertieft: 35517 für die Mount-Optionen von /dev/shm und 35509 für nicht benötigte Dateisystemmodule.

### Check 35517: erfolgreich gehärtet

Die Option noexec wurde für /dev/shm ergänzt. Nach erneutem Einhängen und Neustart bestätigte findmnt die Optionen rw,nosuid,nodev,noexec,relatime. Der Check wechselte auf Passed und die Persistenz wurde geprüft.




*Vorher: Check 35517 nicht erfüllt*




*Nachher: Check 35517 nach Neustart bestanden*

### Check 35509: begründete Ausnahme

Nicht benötigte Module wurden blockiert. FUSE blieb aktiv, weil der Ubuntu-Desktop es über GVFS benötigt. Das Modul fat war im Kernel fest eingebaut und konnte nicht wie ein ladbares Modul entladen werden. Da die Policy alle Teilbedingungen verlangt, blieb der Gesamtcheck auf Failed. Die Entscheidung wurde als Risk Acceptance dokumentiert, statt die Policy nur für einen besseren Prozentwert zu verändern.

**Security Judgment** Ein grüner Compliance-Wert ist kein Selbstzweck. Die technische Abhängigkeit, die Betriebswirkung und das Restrisiko müssen gemeinsam bewertet werden.

# 9. Sysmon und Encoded PowerShell

## Neue Detection-Kette auf dem Windows-11-Endpunkt ham1

### Ziel und Ergebnis

Am 14.10.2026 wurde Sysmon auf ham1 und DC-Test installiert und in die Wazuh-Überwachung aufgenommen. Der vollständige Detection-Test wurde auf ham1 durchgeführt. Ein ungefährlicher Base64-codierter PowerShell-Aufruf gab nur den Text SOC-Test aus. Sysmon erfasste den Prozessstart, Wazuh verarbeitete das Ereignis und die integrierte Regel 92057 erzeugte einen Level-12-Alert.

| Prüfpunkt | Bestätigtes Ergebnis |
|---|---|
| Endpunkt | ham1 / Agent 002 / 192.168.20.5 |
| Datenquelle | Microsoft-Windows-Sysmon/Operational |
| Lokales Ereignis | Sysmon Event ID 1 mit powershell.exe und -EncodedCommand |
| MITRE-Zuordnung | T1059.001 – PowerShell |
| Wazuh-Erkennung | Rule 92057, Level 12 |
| Alert-Zeit | 14.10.2026, 13:21:55 Uhr |
| Regelentscheidung | Vorhandene Regel verwendet; keine doppelte lokale Regel erstellt |

### Validierte Verarbeitungskette

1. Der Wazuh Agent liest den Sysmon-Ereigniskanal ein.
2. Sysmon Event ID 1 enthält Prozess, Kommandozeile und MITRE-Tag.
3. Der Manager verarbeitet das Ereignis vollständig und ordnet es Agent 002 zu.
4. Die integrierte Regel 92057 erzeugt den Alert im Dashboard.
5. Die temporär aktivierte JSON-Gesamtprotokollierung wurde nach der Kontrolle wieder deaktiviert.

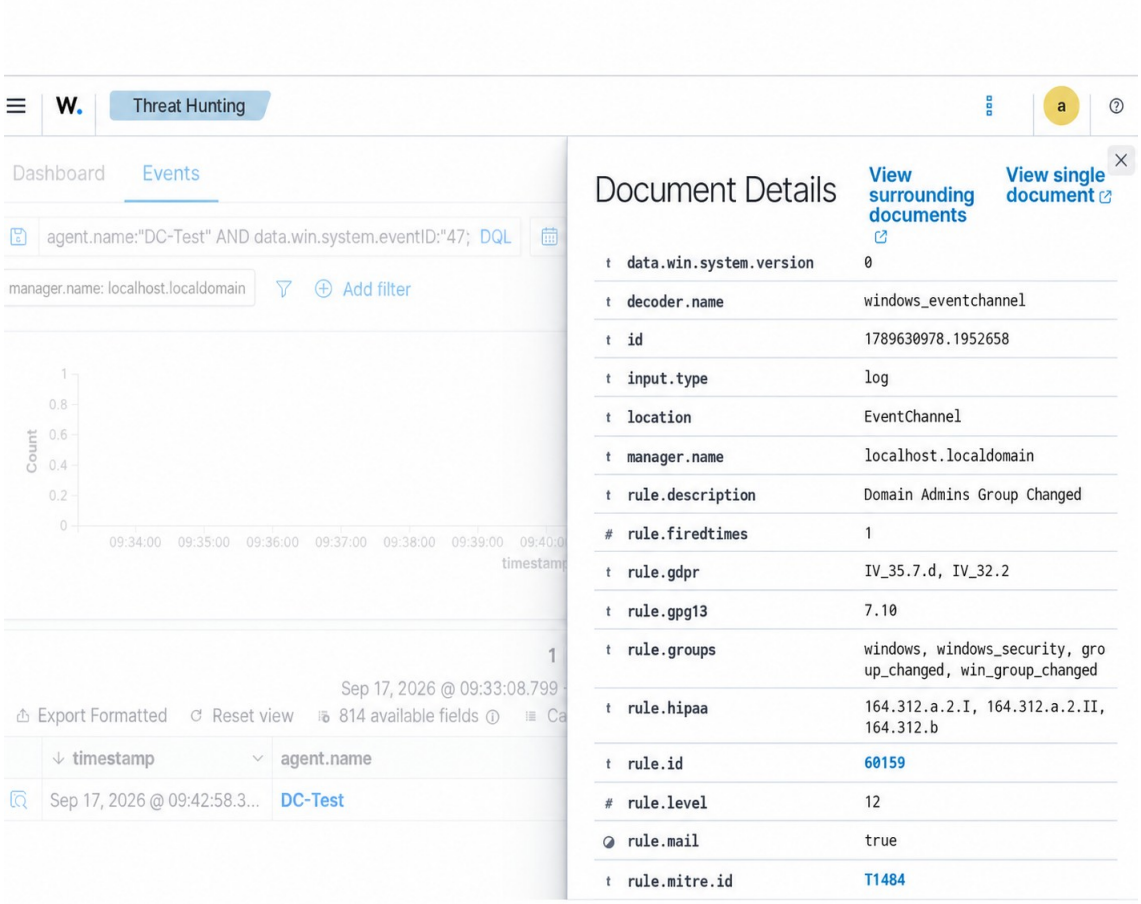



*Abbildung 9: Wazuh Rule 92057 auf Level 12 mit MITRE T1059.001*

## Technische Nachweise

| Nachweisdatei | Inhalt |
|---|---|
| Windows11_Sysmon_EventID1_EncodedCommand | Lokaler Sysmon-Prozessstart mit EncodedCommand |
| Wazuh_Sysmon_EventID1_Document_Details_T1059_001 | Vollständiger Datensatz und MITRE-Zuordnung |
| Wazuh_PowerShell_EncodedCommand_Rule_92057_Level12 | Dashboard-Alert der Regel 92057 auf Level 12 |

**Detection-Entscheidung** Eine eigene Regel wäre technisch möglich, aber fachlich unnötig. Die vorhandene Wazuh-Regel deckt den Use Case bereits ab. Das vermeidet doppelte Alerts und unnötigen Wartungsaufwand.

# 10. Alert Triage und Incident Response

## Vom Treffer zur begründeten Entscheidung

### Triage-Ablauf

1. Alert validieren: richtiger Agent, richtige Zeit, passende Event- oder Rule-ID.
2. Kontext sammeln: Benutzer, Quelladresse, Ziel, Prozess, Kommandozeile und Folgeaktivität.
3. Autorisierung prüfen: geplanter Test, Admin-Tätigkeit oder unbekannte Aktivität.
4. Klassifizieren: True Positive, False Positive, benign true positive oder Incident.
5. Reaktion wählen: dokumentieren, weiter untersuchen, eindämmen oder eskalieren.
6. Abschluss kontrollieren: Bereinigung, erneuter Test und Lessons Learned.

### Kurz-Runbooks

| Alerttyp | Wichtige Prüfungen | Mögliche Reaktion |
|---|---|---|
| Network Scan | Quelle, Zielports, Zeitfenster, Genehmigung, Folgeaktivität | Bei unbekannter Quelle eskalieren und Zugriff begrenzen |
| Fehlgeschlagene Anmeldung | Benutzer, Quell-IP, Häufigkeit, erfolgreiche Folgeanmeldung | Konto und Quelle prüfen; bei Muster sperren oder eskalieren |
| Privilegierte AD-Gruppe | 4728/4729, ausführendes Konto, Change-Freigabe | Unberechtigte Mitgliedschaft entfernen und Konto absichern |
| PowerShell / Sysmon | Kommandozeile, Parent Process, Benutzer, Host, Netzwerkaktivität | Bei unbekannter Aktivität Prozesskette und Persistenz untersuchen |
| FIM | Pfad, Aktion, Hash/Diff, Benutzer, genehmigte Änderung | Datei sichern, Ursprung klären, bei Manipulation isolieren |

### Beispiel: Nmap-Alert

Regel 100111 wurde als True Positive und autorisierte Aktivität bewertet: Quelle 192.168.20.7 war das bekannte Kali-Testsystem, Ziel war DC-Test und das Zeitfenster war geplant. Deshalb war keine Isolation notwendig. Derselbe Alert aus unbekannter Quelle hätte eine Untersuchung und mögliche Eindämmung ausgelöst.

# 11. Ergebnisse, Grenzen und Lessons Learned

## Technische Stärke zeigen, ohne Laborgrenzen zu verschweigen

### Erreichte Ergebnisse

- Zentrale Überwachung von Windows Server, Windows 11 und Ubuntu in einem gemeinsamen SIEM.
- Erfolgreiche Standarderkennungen sowie selbst entwickelte und getestete Korrelation für den Netzwerkscan.
- Nachvollziehbare MITRE-Zuordnung für T1046 und T1059.001.
- Dokumentierte Analystenentscheidungen statt rein technischer Alert-Sammlung.
- Saubere Abschlusskontrollen und Entfernung temporärer Testobjekte.

### Aktuelle Grenzen

| Grenze | Bedeutung | Geplante Verbesserung |
|---|---|---|
| Kleine Laborumgebung | Keine Aussage zu produktiver Last oder 24/7-Betrieb | Messbare KPIs und längere Testzeiträume |
| Ein einzelner Suricata-Sensor | Der Sensor sieht den Verkehr zur Ubuntu-VM, aber keinen vollständigen virtuellen Switch-Traffic. | Später dedizierten Sensor oder kontrolliertes Port Mirroring im reinen Labornetz prüfen. |
| Begrenzte Detection-Sammlung | Nur ausgewählte Use Cases getestet | Zusätzliche Sigma-Regeln und realistische Angriff-und-Verteidigungsszenarien |
| Keine Hochverfügbarkeit | Single Points of Failure bleiben bestehen | Backup-, Restore- und Betriebsdokumentation |
| Screenshots enthalten Labordaten | Vor öffentlicher Veröffentlichung prüfen | Redaktion und sichere GitHub-Auswahl |

### Lessons Learned

- Ein Event ist noch keine Detection; eine Detection ist noch kein Incident.
- Bei fehlenden Alerts müssen Auditierung, Transport, Decoding und Regelwerk getrennt geprüft werden.
- Feste Zeitbasis, eindeutige Agentnamen und stabile Adressen sparen viel Analysezeit.
- Eigene Regeln brauchen Validierung, kontrollierte Tests und Schutz vor doppelten Alerts.
- Risk Acceptance ist besser als eine kosmetische Änderung, die den Betrieb gefährdet.

# 12. Portfolio-Struktur für Bewerbungen

Dokumentation als Einstiegspunkt, technische Artefakte separat und prüfbar

### Empfohlene GitHub-Struktur

| Pfad | Inhalt |
|---|---|
| README.md | Kurzbeschreibung, Architektur, wichtigste Ergebnisse und Screenshots |
| docs/portfolio | Diese vollständige Projektdokumentation als PDF |
| docs/evidence | Ausgewählte, geprüfte und ggf. geschwärzte Nachweise |
| rules/wazuh | Lokale Regeln 100110, 100111, 100120 und 100121 mit Erklärung und Testbezug |
| configs | Bereinigte Konfigurationsauszüge für Agents, Sysmon und SCA |
| runbooks | Kurze Runbooks für Scan, Login, AD, PowerShell und FIM |
| tests | Reproduzierbare Testfälle, erwartetes Ergebnis und tatsächlicher Alert |
| rules/sigma | Validierte Sigma-Regeln mit Status, Logsource, Detection und Testnotiz |
| rules/suricata | Lokale Suricata-Regeln 1000001 und 1000002 inklusive Tuning |

### Umgang mit Befehlen

Die Portfolio-Dokumentation zeigt nur kurze, erklärungsrelevante technische Angaben. Vollständige Befehle und Konfigurationen gehören in separate, versionierte Dateien im Repository. Dadurch bleibt das Dokument lesbar, während Recruiter und technische Interviewer die Umsetzung trotzdem reproduzieren können.

**Veröffentlichungscheck** Vor dem Upload werden Kennwörter, Tokens, interne Daten und nicht benötigte Benutzernamen entfernt. Nur im Labor getestete Befehle und Regeln werden veröffentlicht.

## 13. Entwicklungsroadmap

Vom funktionierenden Labor zum überzeugenden SOC-Portfolio

| Phase | Ziel | Status |
|---|---|---|
| 1. Wazuh + Sysmon + Sigma | Sysmon auf ham1 und DC-Test; Sigma-Regel erstellt und validiert | Abgeschlossen |
| 2. Suricata | Netzwerk-Telemetrie integriert, Scan erkannt, Alarmflut reduziert und Wazuh-Regel 100121 erstellt | Abgeschlossen |
| 3. Angriff/Verteidigung | Realistische, ungefährliche Szenarien reproduzierbar testen | Nächster Schwerpunkt |
| 4. Dashboards und IR | Triage, Priorisierung, Kennzahlen und Playbooks verbessern | Runbook-Basis vorhanden |
| 5. GitHub Portfolio | Regeln, Tests, Nachweise und Dokumentation professionell veröffentlichen | Vorbereitung |
| 6. Bewerbung | CV-Projekttext und Interviewfragen aus dem Projekt ableiten | Danach |

# 14. Nachweisregister

## Belege aus dem gesamten Projektstand bis 17.10.2026

### Kernnachweise im Dokument

| Nr. | Nachweis |
|---|---|
| 01 | Architektur und Datenfluss |
| 02 | Drei aktive Wazuh Agents |
| 03 | Ubuntu FIM mit Inhaltsänderung |
| Abgeschlossen | Windows Event 4625 auf ham1 |
| Abgeschlossen | Rule 60159 für Domain Admins |
| Nächster Schwerpunkt | Rule 100111 für Network Scan, Level 10 |
| 07 | SCA Check 35517 vor der Härtung |
| 08 | SCA Check 35517 nach Neustart Passed |
| 09 | Wazuh-Dienste active |
| 10 | Regelvalidierung Exit Code 0 |
| 11 | AD-Testkonto entfernt |
| 12 | Sysmon Event ID 1 mit EncodedCommand |
| 13 | Wazuh-Dokumentdetails mit MITRE T1059.001 |
| 14 | Wazuh Rule 92057, Level 12 |
| 15 | Sysmon Event ID 1 für whoami /priv auf DC-Test |
| 16 | Sigma-Regel proc_creation_win_whoami_priv ohne Syntax- und Condition-Fehler |
| 17 | Suricata TCP-SYN-Erkennung vor Tuning mit 205 Wazuh-Treffern |
| 18 | Suricata TCP-SYN-Erkennung nach Tuning mit einem Wazuh-Treffer |
| 19 | Wazuh Rule 100121, Level 8, MITRE T1046 |
| 20 | Wazuh Dashboard: Rule 100121 mit zwei Treffern |

### Dokumentationsprinzip

Die Nachweise zeigen nur erfolgreiche, relevante Ergebnisse. Fehlversuche wurden für die Problemlösung genutzt, aber nicht als unnötige Schritt-für-Schritt-Historie aufgenommen. Wo ein Fehler zu einer wichtigen technischen Erkenntnis führte – zum Beispiel beim Nmap-Detection-Gap oder bei der SCA-Ausnahme – ist die Ursache mit der endgültigen Lösung dokumentiert.

# 15. Detection Engineering mit Sysmon, Sigma und Suricata

## Sysmon und Sigma abschließen sowie Suricata als neue Netzwerkdatenquelle integrieren

### 15.1 Ziel und Gesamtergebnis

Dieser Projektabschnitt verbindet hostbasierte Prozessüberwachung, portable Detection-Logik und netzwerkbasierte Telemetrie zu einem durchgängigen Analyseablauf. Sysmon erfasste einen kontrollierten whoami /priv-Prozessstart auf DC-Test. Die zugehörige Sigma-Regel wurde mit Sigma CLI geprüft. Suricata wurde auf ubuntu-Desktop installiert, in Wazuh eingebunden und mit einem autorisierten SYN-Scan von Kali validiert. Durch Threshold-Tuning wurde die Trefferzahl von 205 auf einen aussagekräftigen Alert reduziert. Eine eigene Wazuh-Regel hob den finalen Treffer auf Level 8 an und ordnete ihn MITRE T1046 zu.

| Baustein | Ergebnis |
|---|---|
| Sysmon | Process Create auf DC-Test für whoami /priv erfolgreich erfasst |
| Sigma | Regel proc_creation_win_whoami_priv.yml technisch validiert |
| Suricata | ICMP und TCP-SYN-Aktivität in eve.json erkannt |
| Wazuh-Integration | Suricata-JSON als lokale Logquelle verarbeitet |
| Alert-Tuning | 205 gleichartige Treffer auf einen Alert pro Quelle und Zeitfenster reduziert |
| MITRE ATT&CK | Finale Regel 100121 auf Level 8 mit T1046 Network Service Discovery |

### 15.2 Sysmon und Sigma

Sysmon war auf Windows 11 und Windows Server installiert. Für den neuen Test wurde auf DC-Test der ungefährliche Befehl whoami /priv ausgeführt. Sysmon Event ID 1 lieferte Prozessname, vollständige Kommandozeile, Benutzer- und Elternprozesskontext. Die bestehende Wazuh-Regel 100120 bestätigte die Ende-zu-Ende-Verarbeitung. Danach wurde die Detection als Sigma-Regel gespeichert, damit die Logik verständlich, versionierbar und später auf andere SIEM-Plattformen übertragbar bleibt.

Wichtige Prüf- und Testbefehle:

```
whoami /priv

Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10 |
  Where-Object {$_.Message -match "whoami.exe"} |
  Select-Object -First 1 TimeCreated, Id, Message

sigma check ~/soc-portfolio/rules/sigma/proc_creation_win_whoami_priv.yml
```

Validierungsergebnis  Die Sigma-Prüfung meldete keine Rule Errors und keine Condition Errors. Der Regelstatus wurde passend für eine neu entwickelte Laborregel als experimental geführt. Damit ist die Regel technisch sauber, ohne sie als bereits produktionsreif darzustellen.

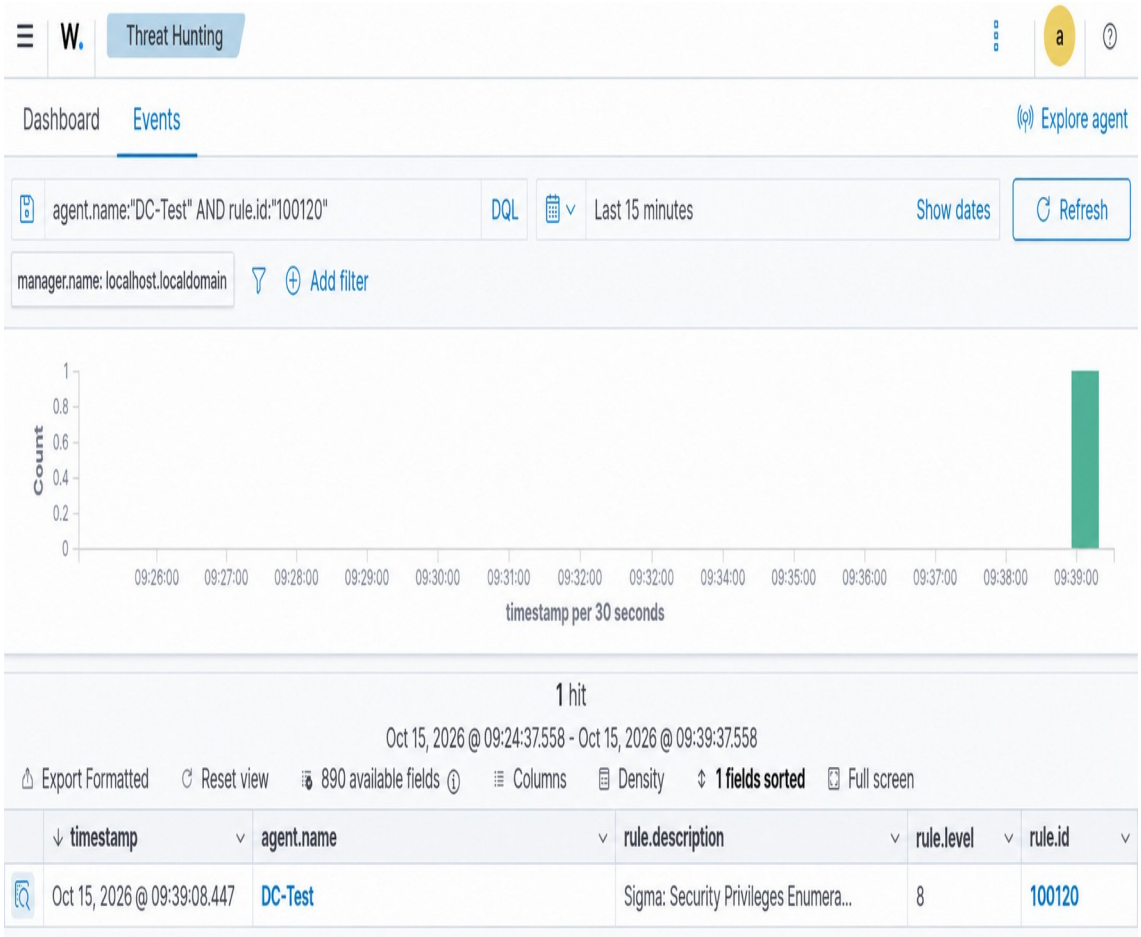



*Abbildung 10: Wazuh Rule 100120 für whoami /priv auf DC-Test*

## 15.3 Suricata installieren und konfigurieren

Suricata wurde direkt auf ubuntu-Desktop installiert. Der Sensor überwacht eth0 mit der Adresse 192.168.20.20. HOME_NET wurde auf das interne Labornetz 192.168.20.0/24 begrenzt. Der physische Hyper-V-Host wurde nicht verändert und nicht als Angriffsziel verwendet.

Installation und Grundprüfung:

```
sudo apt-get install software-properties-common -y
sudo add-apt-repository ppa:oisf/suricata-stable -y
sudo apt-get update
sudo apt-get install suricata jq -y

sudo suricata-update
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo systemctl restart suricata
sudo systemctl is-active suricata
```

| Konfiguration | Wert |
|---|---|
| HOME_NET | [192.168.20.0/24] |
| Capture-Interface | eth0 |
| Event-Ausgabe | /var/log/suricata/eve.json |
| Standardregeln | /var/lib/suricata/rules/suricata.rules |
| Lokale Regeln | /var/lib/suricata/rules/local.rules |

## 15.4 Suricata mit Wazuh verbinden

Der Wazuh Agent auf Ubuntu liest eve.json als JSON-Logquelle. Dadurch bleiben die Suricata-Felder wie Quelladresse, Zieladresse, Signatur-ID, Signaturtext und Severity strukturiert durchsuchbar.

```
<localfile>
 <log_format>json</log_format>
 <location>/var/log/suricata/eve.json</location>
</localfile>
```

```
sudo systemctl restart wazuh-agent
sudo systemctl is-active wazuh-agent
```

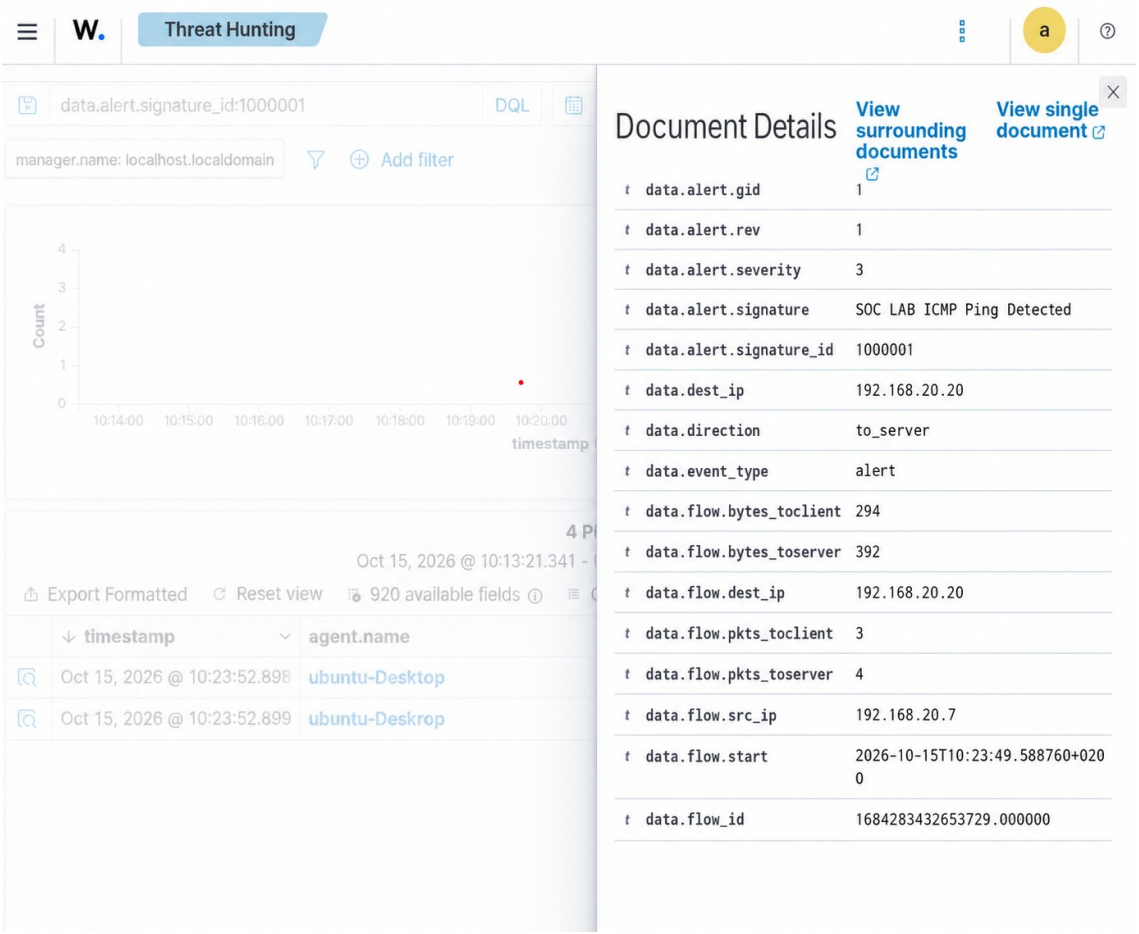



*Abbildung 11: Suricata ICMP SID 1000001 strukturiert in Wazuh*

## 15.5 Lokale Suricata-Regeln und kontrollierte Tests

Zuerst bestätigte eine ICMP-Regel mit SID 1000001, dass Paketerfassung, Suricata-Auswertung und Wazuh-Weiterleitung funktionieren. Für den Scan-Use-Case wurde danach SID 1000002 erstellt. Die Regel erkennt TCP-SYN-Pakete gegen Ubuntu und begrenzt die Ausgabe auf einen Alert pro Quelladresse innerhalb von zehn Sekunden.

```
alert icmp any any -> 192.168.20.20 any (msg:"SOC LAB ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
```

```
alert tcp any any -> 192.168.20.20 any (msg:"SOC LAB Possible TCP SYN Port Scan"; flags:S; flow:stateless; threshold:type both, track by_src, count 20, seconds 10; classtype:attempted-recon; sid:1000002; rev:1;)
```

Autorisierter Test auf Kali Linux:

```
sudo nmap -sS -Pn -T3 -p 1-1000 192.168.20.20
```

Lokale Prüfung auf Ubuntu:

```
sudo jq -c 'select(.event_type=="alert" and .alert.signature_id==1000002) |
{timestamp,src_ip,dest_ip,dest_port,signature_id:.alert.signature_id,signature:.alert.signature,severity:.alert.
severity}' /var/log/suricata/eve.json | tail -n 5
```

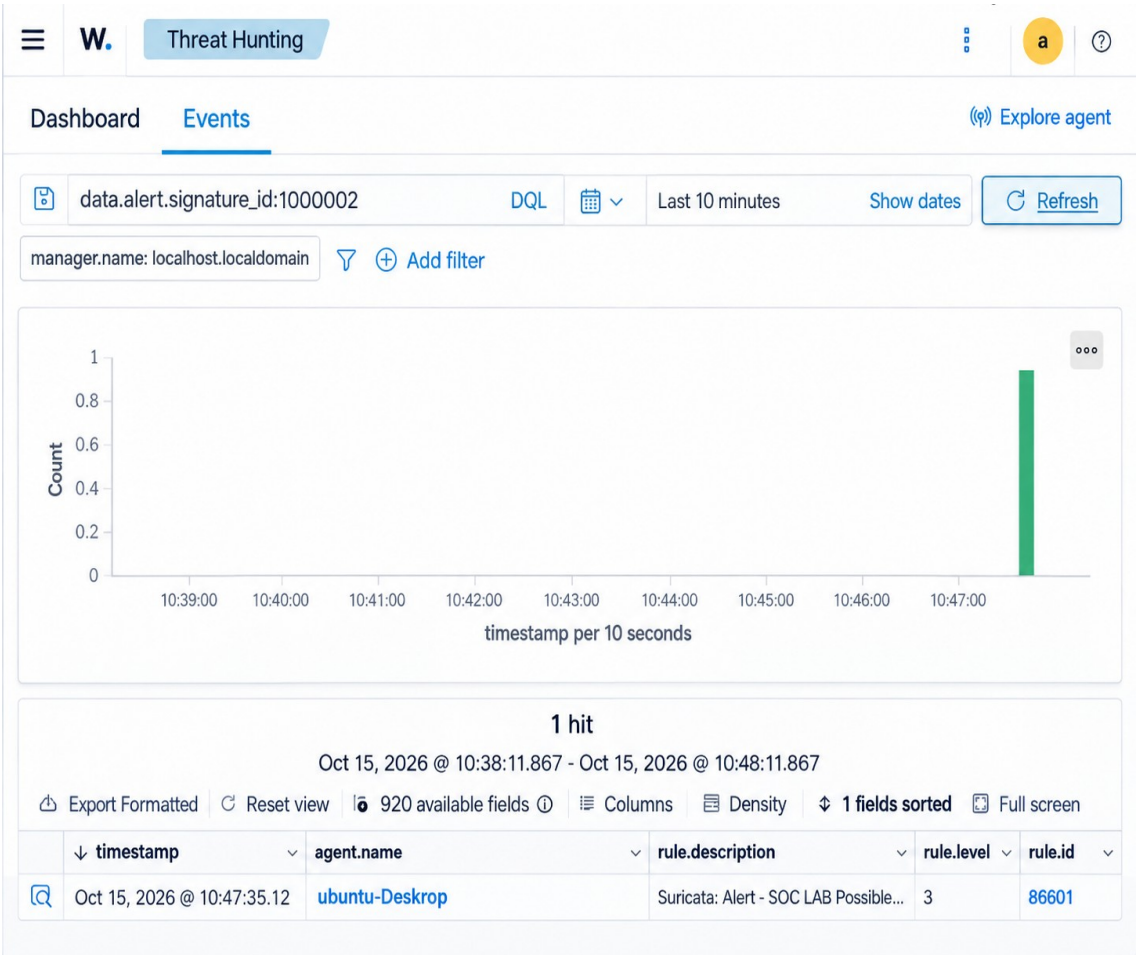



*Abbildung 12: Nach dem Tuning bleibt ein Wazuh-Treffer für Suricata SID 1000002*

## 15.6 Wazuh-Regel und MITRE-Zuordnung

Die allgemeine Suricata-Regel 86601 erzeugte zunächst einen Level-3-Alert. Für den projektspezifischen Scan-Use-Case wurde auf dem Manager die Regel 100121 ergänzt. Sie übernimmt ausschließlich Suricata-SID 1000002, hebt den Alert auf Level 8 an und ordnet das Verhalten MITRE ATT&CK T1046 Network Service Discovery zu.

```
<group name="suricata,network_scan,">
 <rule id="100121" level="8">
  <if_sid>86601</if_sid>
  <field name="alert.signature_id">^1000002$</field>
  <description>Suricata: TCP SYN port scan detected against Ubuntu sensor</description>
  <mitre><id>T1046</id></mitre>
 </rule>
</group>
```

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

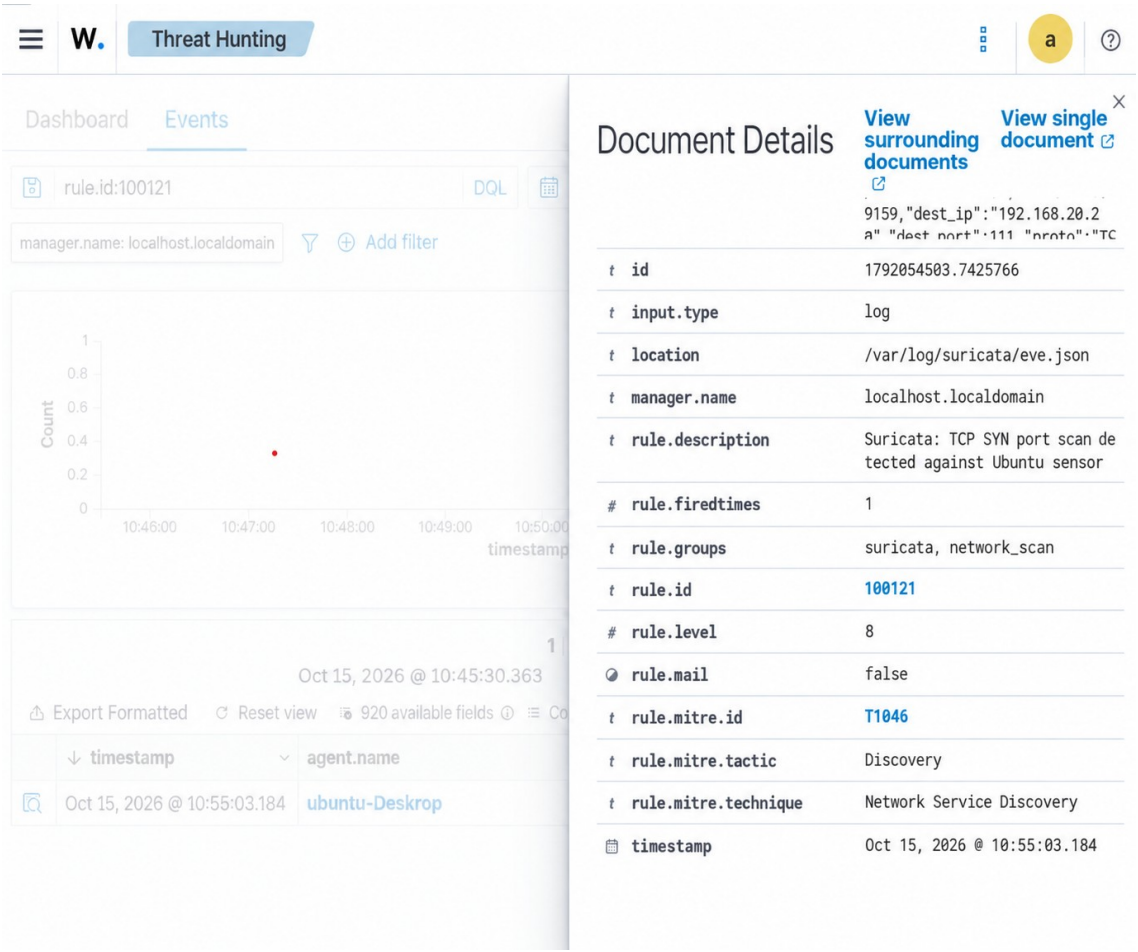



*Abbildung 13: Finale Wazuh Rule 100121 auf Level 8 mit MITRE T1046*

## 15.7 Analystenbewertung und nächste Phase

Der Alert ist ein True Positive, weil der SYN-Scan technisch stattgefunden hat. Er ist zugleich autorisierte Laboraktivität und daher kein Incident. Für die Triage wurden Quelle 192.168.20.7, Ziel 192.168.20.20, Signatur 1000002, Zeitfenster, Rule-ID 100121, Level 8 und MITRE T1046 geprüft. In einer produktiven Umgebung würde dieselbe Aktivität aus einer unbekannten Quelle eine Untersuchung der gescannten Ziele, betroffenen Ports und möglicher Folgeaktivitäten auslösen.

Abschluss der Phasen 3 und 4  Die dokumentierten Fallstudien verbinden kontrollierte Tests, Telemetrie, Alert-Triage, Klassifizierung und Incident-Response-Entscheidungen. Der nächste Schwerpunkt ist die strukturierte Veröffentlichung als GitHub-Portfolio.

### 15.8 Alertanalyse zur Wazuh-Regel 100121

Am 16.10.2026 wurde im Wazuh-Modul Threat Hunting im Dashboard nach rule.id:100121 gesucht. Für den sichtbaren Zeitraum „Last 30 days“ zeigte die Ansicht zwei Alert-Ereignisse. Die Ereignisliste wies einen Treffer vom 15.10.2026 gegen 10:55 Uhr und einen vom 16.10.2026 gegen 08:32 Uhr aus. Diese zwei Dokumente belegen zwei erfasste Alerts; daraus wird nicht abgeleitet, dass zwei getrennte Scanläufe stattgefunden haben.

Die geöffneten Detailansichten bestätigten Agent ubuntu-Desktop (ID 003, 192.168.20.20), Quelle Kali 192.168.20.7, Ziel 192.168.20.20, Protokoll TCP, Suricata-Signatur SOC LAB Possible TCP SYN Port Scan (SID 1000002) und Wazuh-Regel 100121 auf Level 8. Der ältere Alert vom 15.10.2026 erfasste Zielport 111, der neuere vom 16.10.2026 Zielport 199. Damit liegen zwei Alert-Dokumente zu unterschiedlichen Flows vor; daraus lässt sich nicht sicher auf zwei getrennte Scanläufe schließen. Das Feld data.alert.action zeigte in beiden geprüften Details allowed: Suricata erkannte den Verkehr, blockierte ihn jedoch nicht.

| Zeitstempel | Quelle → Ziel | TCP-Zielport | Aktion |
|---|---|---|---|
| **15.10.2026 10:55:03.184** | 192.168.20.7 (Kali) → 192.168.20.20 (Ubuntu) | 111 | allowed – erkannt, nicht blockiert |
| **16.10.2026 08:32:09.683** | 192.168.20.7 (Kali) → 192.168.20.20 (Ubuntu) | 199 | allowed – erkannt, nicht blockiert |

Beide Alerts gehören zu Suricata-SID 1000002 und Wazuh-Regel 100121 (Level 8). Die zwei Alert-Dokumente belegen unterschiedliche Flows, aber nicht sicher zwei getrennte Scanläufe.

Bewertung: Die Detection war erfolgreich. Die Anzeige von null Alerts ab Level 12 ist mit dem Rule Level 8 vereinbar und bedeutet keine fehlgeschlagene Erkennung. Wegen des genehmigten Labortests lautet die Klassifizierung True Positive – Authorized Security Test; eine Eindämmung war nicht erforderlich.

## 16. SOC-Fallstudie Suricata TCP-SYN-Portscan

## Vom kontrollierten SYN-Scan bis zur dokumentierten Analystenentscheidung

### 16.1 Ziel und Ablauf

Diese Fallstudie überführt die bereits validierte Netzwerkerkennung in einen vollständigen SOC-Arbeitsablauf. Ein autorisierter TCP-SYN-Scan von Kali wurde durch Suricata erkannt, strukturiert an Wazuh weitergeleitet, mit MITRE ATT&CK verknüpft und anschließend triagiert. Der Test blieb vollständig innerhalb des isolierten Labornetzes 192.168.20.0/24.

**Detection Chain** Kali Nmap -> Suricata SID 1000002 -> eve.json -> Wazuh Rule 100121 -> MITRE T1046 -> Triage -> Closure.

### 16.2 Falldaten

| Feld | Bewertung |
|---|---|
| Case ID | SOC-2026-001 |
| Titel | Network Discovery detected from Kali |

| Feld | Bewertung |
|---|---|
| Quelle | 192.168.20.7 - Kali Linux |
| Ziel | 192.168.20.20 - Ubuntu / Suricata Sensor |
| Test | sudo nmap -sS -Pn -p 1-1000 --reason 192.168.20.20 |
| Suricata | SID 1000002 - SOC LAB Possible TCP SYN Port Scan |
| Wazuh | Rule 100121, Level 8 |
| MITRE ATT&CK | T1046 - Network Service Discovery |
| Klassifizierung | True Positive - Authorized Security Test |
| Status | Closed |
| Dashboard-Abfrage | rule.id:100121; Last 30 days: 2 Alert-Ereignisse (15. und 16.10.2026); Zielports 111 und 199 |
| Suricata-Aktion | allowed; der erfasste Verkehr wurde nicht blockiert |

### 16.3 Triage und Analystenentscheidung

1. Alert validieren: Regel-ID, Severity, Zeitstempel, Quell- und Zieladresse sowie die Suricata-Signatur wurden geprüft.
2. Quelle einordnen: 192.168.20.7 ist das bekannte Kali-System des isolierten Testlabors.
3. Umfang bestimmen: Der Scan richtete sich ausschließlich gegen 192.168.20.20 und die Ports 1 bis 1000.
4. Kontext prüfen: Zeitpunkt und Scanparameter stimmen mit dem autorisierten Test überein.
5. Begleitaktivität prüfen: Es lagen keine zusätzlichen Hinweise auf Exploitation, Persistenz oder Datenabfluss vor.
6. Entscheidung treffen: Die Detection ist technisch korrekt und daher ein True Positive; der bestätigte Labortest ist jedoch kein Security Incident.

**Analystenbewertung** Der TCP-SYN-Scan hat tatsächlich stattgefunden und wurde korrekt erkannt. Da Quelle, Ziel, Zeitfenster und Zweck des Tests bekannt und genehmigt waren, wurde der Alert als True Positive - Authorized Security Test geschlossen. Ein False Positive wäre fachlich falsch, weil das gemeldete Verhalten real vorlag.

## 16.4 Incident-Response-Runbook

| Phase | Analystenaktion | Eskalationskriterium oder Ergebnis |
|---|---|---|
| 1. Validate | Alertfelder, Zeitstempel, Regel, Signatur und Datenquelle bestätigen. | Ungültige oder unvollständige Daten zur technischen Prüfung zurückgeben. |
| 2. Scope | Quelle, Ziel, Portbereich, Häufigkeit und weitere Alerts derselben Quelle bestimmen. | Bei mehreren Zielen oder wiederholter Aktivität Priorität erhöhen. |
| 3. Enrich | Asset-Rolle, Besitzer, Change- oder Testfreigabe und bekannte Scanner prüfen. | Unbekannte Quelle oder fehlende Freigabe als verdächtig behandeln. |
| 4. Classify | True Positive, False Positive oder autorisierten benignen True Positive festlegen. | Bei unbekannter Motivation als potenziellen Incident eskalieren. |
| 5. Contain | Bei unautorisiertem Scan Quelle isolieren oder blockieren und betroffene Systeme überwachen. | Im bestätigten Labortest war keine Eindämmung erforderlich. |
| 6. Close | Entscheidung, Belege, Maßnahmen und Verbesserungen nachvollziehbar dokumentieren. | Status Closed mit Begründung und Referenz auf die Evidence setzen. |

## 16.5 Abschluss und Verbesserungen

Es war keine Eindämmung erforderlich. Für einen produktiven Betrieb würde dieselbe Detection bei einer unbekannten Quelle, mehreren Zielsystemen oder zusätzlicher Exploit-Aktivität eskaliert. Das bereits umgesetzte Threshold-Tuning reduziert Alert-Duplikate, ohne den eigentlichen Scan zu übersehen. Als Beleg dienen Abbildung 12 für das Tuning-Ergebnis und Abbildung 13 für die finale Wazuh-Regel mit MITRE-Zuordnung. Abschnitt 15.8 enthält die dokumentierte Auswertung der beiden Alert-Ereignisse im 30-Tage-Zeitraum.

**Closure Note** Authorized laboratory scan confirmed. Detection worked as designed. No compromise indicators identified. No containment required. Case closed.

# 17. SOC-Fallstudie Encoded PowerShell

## Von Sysmon-Telemetrie zur kontextbasierten Incident-Entscheidung

### 17.1 Ziel und Ablauf

Diese Fallstudie nutzt den bereits erfolgreich durchgeführten EncodedCommand-Test auf Windows 11. Sysmon erfasste die Prozesskette und die vollständige Befehlszeile. Wazuh erkannte das Base64-kodierte PowerShell-Verhalten mit der integrierten Regel 92057 und ordnete es MITRE ATT&CK T1059.001 zu. Für die Fallstudie wurde der Test nicht erneut ausgeführt; die vorhandene, validierte Evidence wurde analysiert.

**Detection Chain** Windows 11 ham1 -> PowerShell EncodedCommand -> Sysmon Event ID 1 -> Wazuh Rule 92057 -> MITRE T1059.001 -> Triage -> Closure.

### 17.2 Falldaten

| Feld | Bewertung |
|---|---|
| Case ID | SOC-2026-002 |
| Titel | Encoded PowerShell detected on Windows 11 |
| Endpoint | ham1 - Windows 11, 192.168.20.5 |
| Benutzerkontext | Autorisierter lokaler Labortest |
| Datenquelle | Microsoft-Windows-Sysmon/Operational, Event ID 1 |
| Erkennungsmerkmal | powershell.exe mit -EncodedCommand und Base64-Inhalt |
| Wazuh | Rule 92057, Level 12 |
| MITRE ATT&CK | T1059.001 - PowerShell |
| Klassifizierung | True Positive - Authorized Security Test |
| Status | Closed |

### 17.3 Triage und Analystenentscheidung

1. Alert validieren: Host, Benutzer, Zeitstempel, Rule-ID, Level und MITRE-Zuordnung prüfen.
2. Prozesskette prüfen: Image, ParentImage, CommandLine und ausführenden Benutzer aus Sysmon Event ID 1 auswerten.
3. EncodedCommand dekodieren: Den Base64-Inhalt offline als Unicode dekodieren und den tatsächlichen PowerShell-Befehl bewerten, ohne ihn erneut auszuführen.
4. Umfang bestimmen: Nach derselben Befehlszeile, demselben Benutzer und vergleichbaren Alerts auf weiteren Endpunkten suchen.

5. Begleitaktivität prüfen: Netzwerkverbindungen, Child Processes, Dateiänderungen, Persistenz und Credential-Zugriffe im Zeitfenster untersuchen.
6. Entscheidung treffen: Der Testbefehl war autorisiert und ungefährlich; die Detection ist korrekt, aber es liegt kein Security Incident vor.

**Analystenbewertung** Encoded PowerShell ist ein starkes Verdachtsmerkmal, aber allein noch kein Beweis für Schadaktivität. In diesem Fall bestätigten Host, Zeitpunkt, Benutzerkontext und der dekodierte Inhalt den genehmigten Labortest. Der Alert wurde daher als True Positive - Authorized Security Test geschlossen.

## 17.4 Incident-Response-Runbook

| Phase | Analystenaktion | Eskalationskriterium oder Ergebnis |
|---|---|---|
| 1. Validate | Wazuh- und Sysmon-Felder sowie Vollständigkeit der CommandLine bestätigen. | Fehlende Telemetrie zur Sensor- oder Agentenprüfung geben. |
| 2. Decode safely | Base64-Inhalt offline dekodieren; den Inhalt nicht ungeprüft ausführen. | Download, Obfuskation, Credential-Zugriff oder Persistenz sofort eskalieren. |
| 3. Scope | Gleiche Befehle, Benutzer, Hosts, Hashes und Folgeprozesse im Zeitfenster suchen. | Mehrere Endpunkte oder wiederholte Ausführung erhöhen Priorität und Umfang. |
| 4. Enrich | Asset-Kritikalität, Benutzerrolle und Change- oder Testfreigabe prüfen. | Unbekannter Benutzer oder fehlende Freigabe als Incident behandeln. |
| 5. Contain | Bei bestätigter Schadaktivität Host isolieren, Prozess stoppen und Artefakte sichern. | Im autorisierten Test war keine Eindämmung erforderlich. |
| 6. Close | Dekodierten Inhalt, Entscheidung, Evidence und Maßnahmen dokumentieren. | Closed als autorisierter True Positive mit Verweis auf Abbildung 9. |

## 17.5 Abschluss und Verbesserungen

Es war keine Eindämmung erforderlich. In einer produktiven Umgebung würde dieselbe Detection bei unbekanntem Benutzer, verdächtigem dekodiertem Inhalt, externen Netzwerkverbindungen oder Folgeprozessen sofort eskaliert. Die Entscheidung stützt sich nicht nur auf den Alertnamen, sondern auf Prozesskontext, dekodierten Inhalt und korrelierte Aktivitäten. Abbildung 9 dokumentiert Rule 92057 auf Level 12 mit MITRE T1059.001.

**Closure Note** Authorized Encoded PowerShell laboratory test confirmed. Sysmon telemetry and Wazuh detection worked as designed. Decoded content was benign. No compromise indicators identified. Case closed.

# 18. SOC-Fallstudie Konten und privilegierte Gruppen

## Windows-Anmeldeereignisse und Änderungen an Domain Admins gemeinsam bewerten

### 18.1 Ziel und fachliche Abgrenzung

Die Fallstudie fasst bereits validierte Windows-Sicherheitsereignisse zu einem gemeinsamen Triage-Use-Case zusammen. Event 4625 dokumentiert eine fehlgeschlagene Anmeldung, Event 4624 eine erfolgreiche Anmeldung, Event 4728 das Hinzufügen eines Mitglieds zu einer sicherheitsaktivierten globalen Gruppe und Event 4729 dessen Entfernung. Die ursprünglichen Labortests waren autorisierte Einzeltests. Deshalb wird keine unbelegte Angriffskette zwischen den Ereignissen behauptet; gezeigt wird, wie ein SOC solche Signale zeitlich und fachlich korrelieren würde.

**Detection Chain** DC-Test Security Log -> Wazuh Windows Decoder und Rules -> Authentifizierungs- und Gruppenänderungsanalyse -> MITRE T1098.007 -> Triage -> Closure.

### 18.2 Falldaten

| Feld | Bewertung |
|---|---|
| Case ID | SOC-2026-003 |
| Titel | Account logons and Domain Admins membership change |
| System | DC-Test - Windows Server, 192.168.20.2, Domain test.local |
| Event 4625 | Fehlgeschlagene Anmeldung |
| Event 4624 | Erfolgreiche Anmeldung |
| Event 4728 | Mitglied zu einer sicherheitsaktivierten globalen Gruppe hinzugefügt |
| Event 4729 | Mitglied aus einer sicherheitsaktivierten globalen Gruppe entfernt |
| Wazuh | Rule 60159 für die überwachte Änderung an Domain Admins |
| MITRE ATT&CK | T1098.007 - Additional Local or Domain Groups |
| Klassifizierung | True Positive - Authorized Administrative Test |
| Status | Closed |

## 18.3 Triage und Analystenentscheidung

1. Ereignisse validieren: Computer, Zeitstempel, Event-ID, handelndes Konto, Zielkonto und Quelladresse prüfen.
2. Anmeldungen bewerten: Bei 4625 Fehlergrund, Statuscode, Logon Type und Häufigkeit prüfen; bei 4624 Logon Type und Quellgerät bestätigen.
3. Korrelation vorsichtig durchführen: Nur bei passenden Benutzern, Quellen und Zeitfenstern einen Zusammenhang zwischen 4625 und 4624 annehmen.
4. Gruppenänderung untersuchen: Bei 4728 und 4729 handelndes Konto, betroffenes Mitglied und Zielgruppe Domain Admins kontrollieren.
5. Autorisierung prüfen: Change-Freigabe, Administratorzuständigkeit und Zweck der Gruppenmitgliedschaft bestätigen.
6. Umfang bestimmen: Nach weiteren Kontoänderungen, privilegierten Logons und identischen Aktionen auf dem Domain Controller suchen.
7. Entscheidung treffen: Die protokollierten Aktionen fanden tatsächlich statt, waren aber genehmigte Labortests; es liegt kein Security Incident vor.

**Analystenbewertung** Die Events sind True Positives, weil Anmeldungen und Gruppenänderungen real protokolliert wurden. Die Einstufung als autorisierter administrativer Test erfolgte erst nach Prüfung von Benutzer, Zeitpunkt, Zielgruppe und Testkontext. Wiederholte 4625-Events können auf Brute Force hinweisen, werden aber nicht allein aufgrund eines einzelnen Fehlversuchs als MITRE T1110 klassifiziert.

## 18.4 Incident-Response-Runbook

| Phase | Analystenaktion | Eskalationskriterium oder Ergebnis |
|---|---|---|
| 1. Validate | Event-ID, Host, Benutzer, Quelle, Zeit und Wazuh-Regel bestätigen. | Unvollständige Felder oder Zeitabweichungen technisch klären. |
| 2. Correlate | 4625, 4624, 4728 und 4729 nur anhand passender Felder und Zeitfenster verbinden. | Keine Angriffskette aus zeitlich unabhängigen Events konstruieren. |
| 3. Verify change | Domain-Admins-Änderung mit Change-Freigabe und Administrator abstimmen. | Fehlende Freigabe sofort als High Priority eskalieren. |
| 4. Scope | Weitere privilegierte Logons, Konto- und Gruppenänderungen sowie betroffene Systeme suchen. | Mehrere Konten oder Systeme erweitern Incident Scope. |
| 5. Contain | Bei unautorisierter Änderung Mitglied entfernen, Konto sperren und Sessions prüfen. | Im bestätigten Labortest war keine Eindämmung erforderlich. |

| Phase | Analystenaktion | Eskalationskriterium oder Ergebnis |
|---|---|---|
| 6. Recover | Berechtigungen validieren, Kennwörter oder Tokens bei Bedarf zurücksetzen und Monitoring erhöhen. | Normalbetrieb erst nach bestätigter Berechtigungsprüfung. |
| 7. Close | Evidence, Autorisierung, Entscheidung und Maßnahmen nachvollziehbar dokumentieren. | Closed als autorisierter True Positive. |

## 18.5 Abschluss und Verbesserungen

Es war keine Eindämmung erforderlich. Für einen produktiven Betrieb müssen Änderungen an hochprivilegierten Gruppen sofort mit Change-Daten, verantwortlichem Administrator und weiteren Anmeldeereignissen abgeglichen werden. Besonders wichtig ist die Trennung zwischen tatsächlich korrelierten Aktivitäten und lediglich gemeinsam betrachteten Events. Dadurch bleibt die Analyse reproduzierbar und vermeidet unbelegte Schlussfolgerungen.

**Closure Note** Authorized authentication and Domain Admins membership tests confirmed. Windows Security events and Wazuh detection worked as designed. No unauthorized privilege change or compromise indicators identified. Case closed.

# 19. Dashboard-Übersichten und Darstellung

## Wazuh-Übersicht, fokussierte Alarmansichten und SOC-Dashboard

Die folgenden Screenshots dokumentieren die Dashboard-Ansichten für die tägliche Übersicht und die gezielte Triage. Die Gesamtansicht zeigt Kennzahlen und zeitliche Verteilung. Der Filter auf Rule 100121 grenzt die Suche auf zwei erfasste Alerts ein; die Detailansicht zeigt die dazu geprüften Suricata-Felder. Das gespeicherte SOC-Übersichtsboard bündelt Agenten-, Zeit- und Severity-Panels sowie eine eigene Kennzahl für Alerts der Level 12 bis 15.

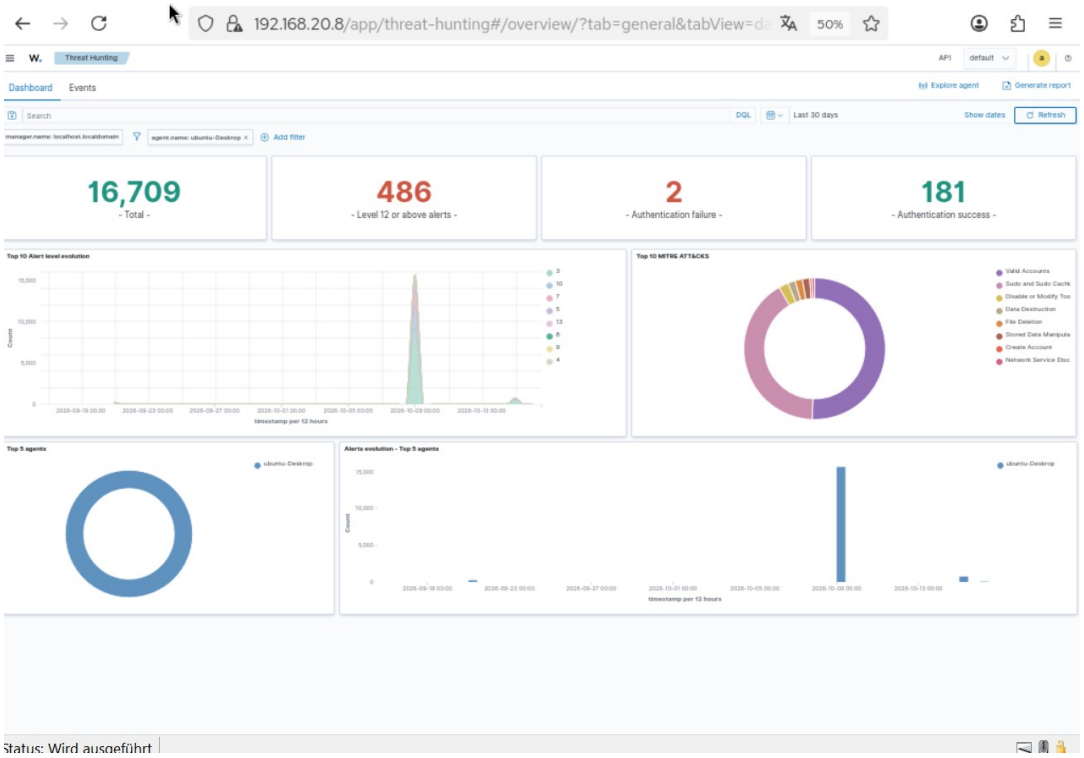



*Abbildung 14: Wazuh Threat-Hunting-Übersicht mit Gesamtzahl, Level-12+-Alerts und Authentifizierungskennzahlen*

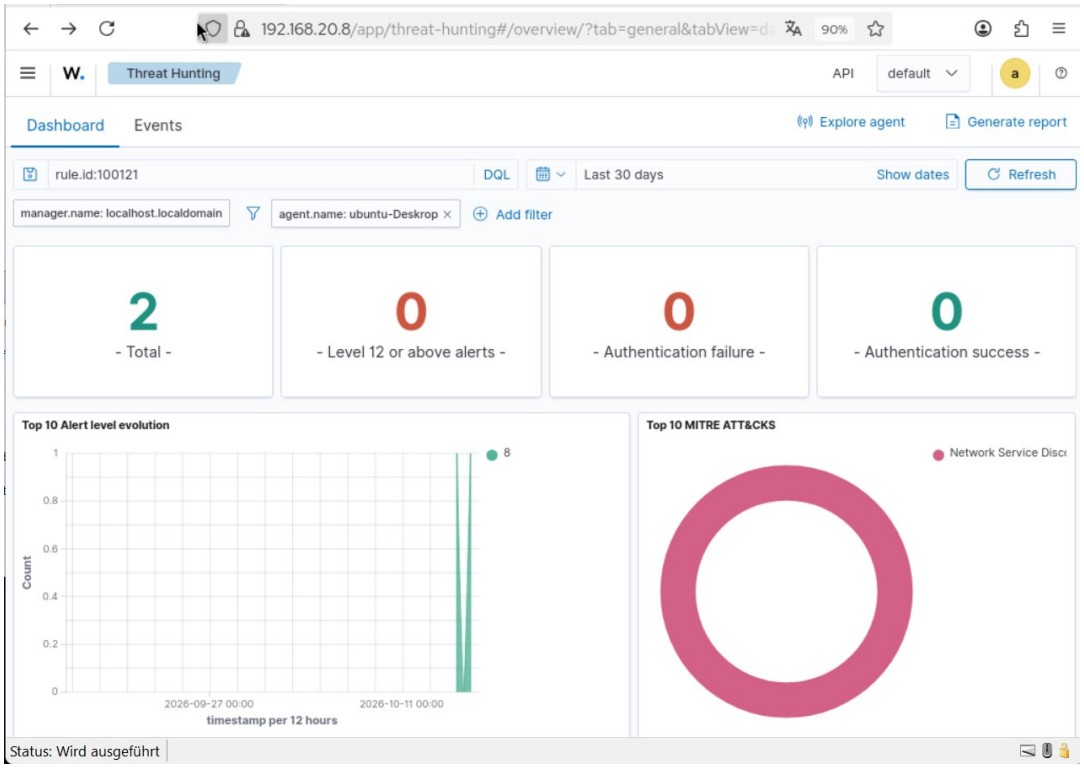



*Abbildung 15: Dashboard mit Rule-ID 100121 und zwei gefundenen Alerts*

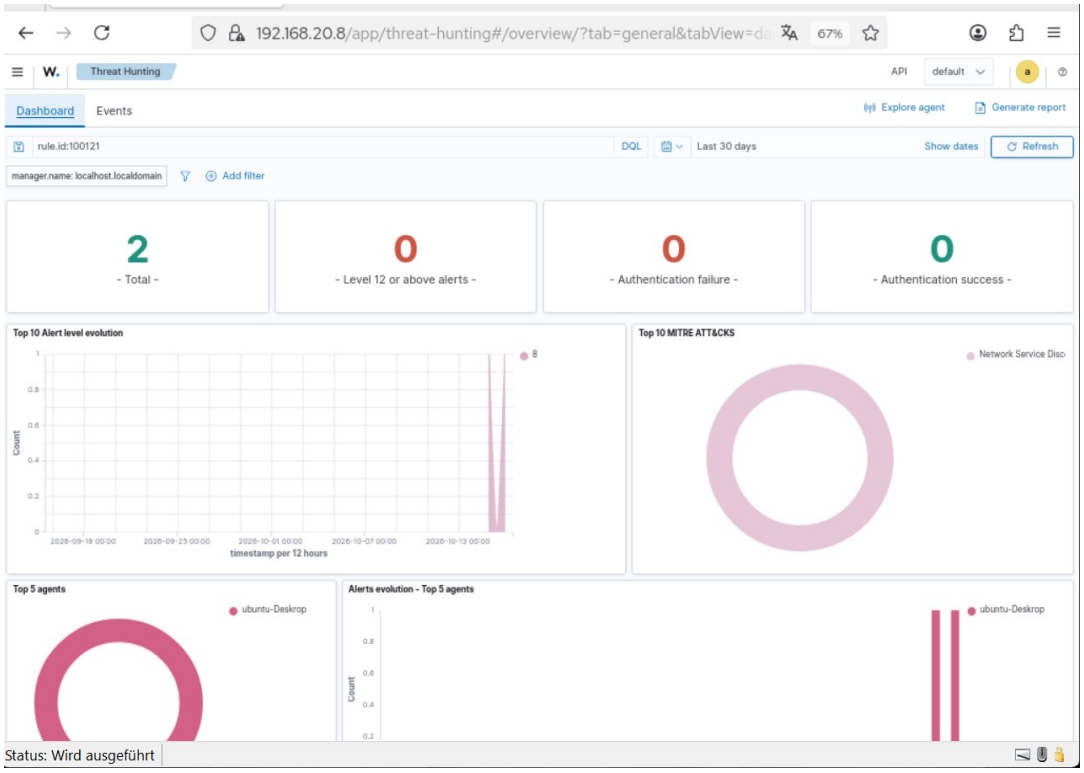



*Abbildung 16: Gefilterte Rule-100121-Ansicht mit Kennzahlen und Alertverteilung*

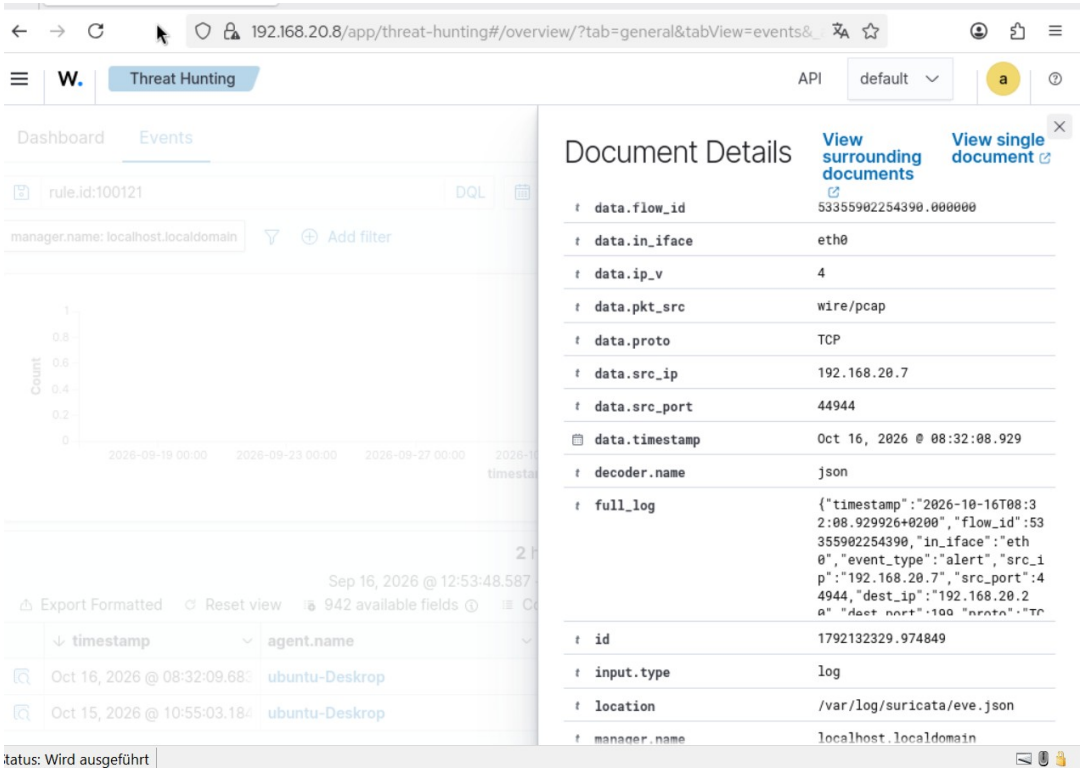



*Abbildung 17: Detailfelder eines Suricata-TCP-SYN-Scan-Alerts in Wazuh*

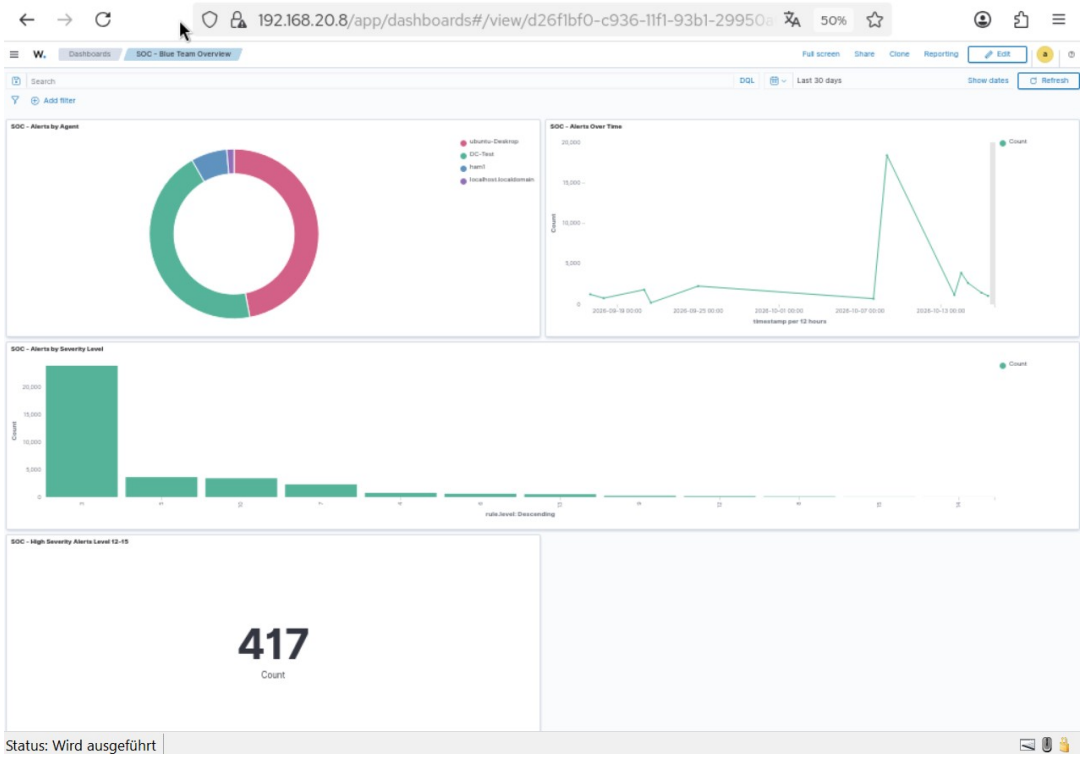



*Abbildung 18: Gespeichertes SOC-Übersichtsboard mit Agenten-, Zeit- und Severity-Auswertung sowie 417 Alerts der Level 12 bis 15*

**Ergebnis der Dashboard-Optimierung** Das Board ist gespeichert und nutzt den Zeitraum „Last 30 days“. Die zusätzliche Kennzahl „SOC - High Severity Alerts Level 12-15“ zeigt 417 priorisierte Alerts und unterstützt die schnelle Auswahl relevanter Ereignisse für die anschließende Triage.

## 19.1 Priorisierte Triage-Übersicht

Das SOC-Übersichtsboard wurde um zwei Auswertungen für die tägliche Alert-Triage erweitert. Die Tabelle „SOC - Top 10 Alert Rules“ zeigt die am häufigsten ausgelösten Regeln. Die darunterliegende Tabelle „SOC - High Severity Triage Summary“ beschränkt die Auswertung auf die Wazuh-Level 12 bis 15 und gruppiert die Treffer nach Schweregrad, Rule-ID, Regelbeschreibung und Anzahl.

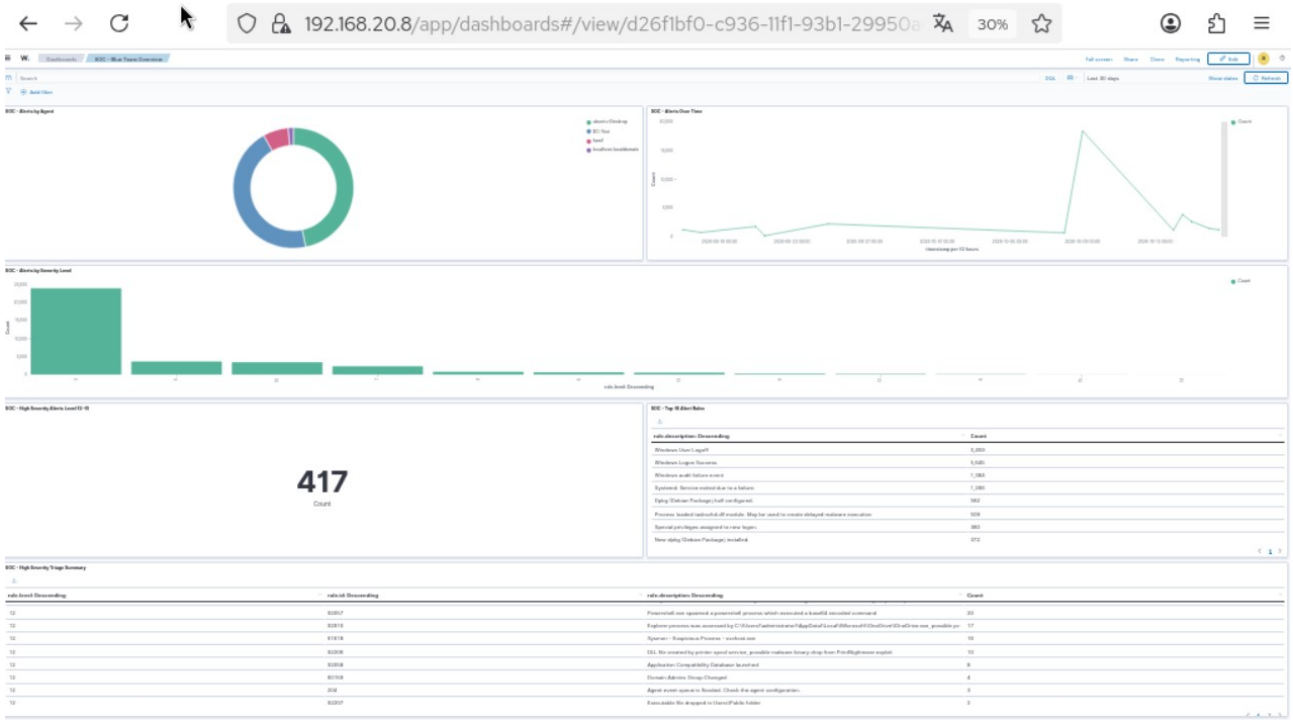



*Abbildung 19: Gespeichertes SOC-Dashboard mit Top-10-Regeln und priorisierter Triage-Zusammenfassung für Alerts der Level 12 bis 15*

**Ergebnis der Triage-Optimierung** Die kombinierte Ansicht ermöglicht eine schnelle Priorisierung: Rule-Level kennzeichnet die Schwere, die Rule-ID macht die Untersuchung reproduzierbar, die Beschreibung liefert den fachlichen Kontext und der Count hebt häufig auftretende Alerts hervor. Damit kann ein Analyst auffällige Regeln auswählen und anschließend in Threat Hunting bis zum Einzelereignis untersuchen.

## 19.2 Triage eines PowerShell-Alerts

Ein vorhandener High-Severity-Alert auf dem Windows-11-Endpunkt ham1 wurde im Threat-Hunting-Modul bis auf Ereignisebene untersucht. Ziel war nicht die erneute Ausführung eines Tests, sondern die nachvollziehbare Bewertung eines bereits erkannten PowerShell-Ereignisses nach dem SOC-Triage-Ablauf.

| Prüfpunkt | Feststellung |
|---|---|
| Endpunkt / Benutzer | ham1 / TEST\Administrator |
| Datenquelle | Sysmon Event ID 1 - Process Creation |
| Wazuh-Erkennung | Rule ID 92057, Level 12 |
| Erkannter Vorgang | PowerShell mit Base64-codiertem Befehl |
| Decodierter Inhalt | Write-Output "SOC-Test" |
| MITRE ATT&CK | T1059.001 - PowerShell; Taktik: Execution |
| Bewertung | True Positive - Authorized Test |
| Reaktion | Keine Eindämmung erforderlich; analytisch mit Begründung geschlossen |

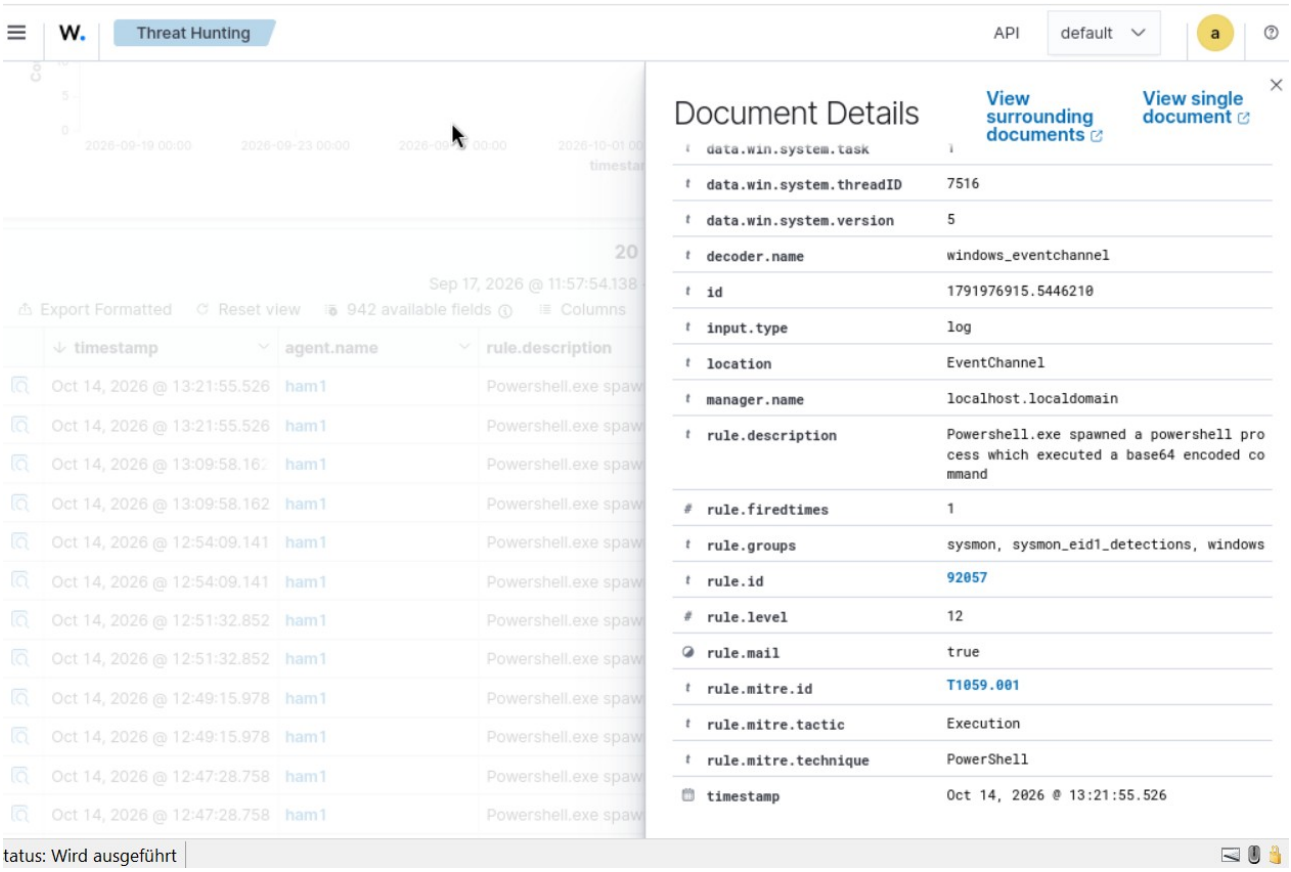



*Abbildung 20: Wazuh-Triage-Nachweis für Rule 92057 mit Level 12, MITRE ATT&CK T1059.001 und der Taktik Execution*

**Triage-Entscheidung** Das Ereignis ist ein True Positive, da der codierte PowerShell-Prozess tatsächlich ausgeführt und durch Sysmon sowie Wazuh korrekt erkannt wurde. Der decodierte Inhalt war jedoch ein ungefährlicher, zuvor autorisierter Labortest. Es gibt keine Hinweise auf Kompromittierung oder unautorisierte Aktivität. Deshalb waren weder Isolation noch weitere Eindämmungsmaßnahmen erforderlich.

**Closure Note** Authorized PowerShell Base64 test confirmed on endpoint ham1. Sysmon Event ID 1 and Wazuh Rule 92057 (Level 12) detected the activity as designed. Decoded content was Write-Output "SOC-Test". No malicious follow-up activity identified; no containment required.

## 19.3 Triage eines Suricata-Netzwerkalerts

Der bereits dokumentierte Suricata-Alert wurde als zweiter Triage-Fall auf Netzwerkebene untersucht. Grundlage waren die zwei Treffer zu Rule 100121 sowie die in den Abbildungen 15 bis 17 dargestellten Dashboard- und Ereignisdetails. Es wurde kein neuer Scan ausgeführt.

| Prüfpunkt | Feststellung |
|---|---|
| Zeitpunkt | 16.10.2026, ca. 08:32 Uhr |
| Quelle | 192.168.20.7 - Kali |
| Ziel | 192.168.20.20 - Ubuntu |
| Netzwerkziel | TCP-Port 199 |
| Suricata-Signatur | SOC LAB Possible TCP SYN Port Scan |
| Suricata-Bewertung | Severity 2; Action: allowed |
| Wazuh-Erkennung | Rule ID 100121, Level 8 |
| MITRE ATT&CK | T1046 - Network Service Discovery |
| Bewertung | True Positive - Authorized Network Scan |
| Reaktion | Keine Eindämmung erforderlich; analytisch geschlossen |

**Analystenbewertung** Die Netzwerkaktivität wurde tatsächlich erzeugt und von Suricata korrekt als möglicher TCP-SYN-Portscan erkannt. Der Quellhost 192.168.20.7 ist das bekannte Kali-System des isolierten Labors. Ziel und Zeitpunkt stimmen mit dem autorisierten Detection-Test überein. Damit ist der Alert ein True Positive, jedoch kein Security Incident.

**Sensorwirkung** Der Wert action=allowed bestätigt den IDS-Betrieb: Suricata erkannte und protokollierte den Verkehr, blockierte ihn aber nicht. Dieses Verhalten entspricht der vorgesehenen Monitoring-Funktion des Labors.

**Closure Note** Authorized TCP SYN scan from Kali 192.168.20.7 to Ubuntu 192.168.20.20 confirmed. Suricata and Wazuh Rule 100121 detected the activity as designed. No unauthorized source, follow-up compromise or containment requirement identified.

## 19.4 Triage einer Domain-Admins-Änderung

Als dritter Triage-Fall wurde eine privilegierte Gruppenänderung auf dem Domain Controller DC-Test untersucht. Der vorhandene Alert zu Rule 60159 wurde über einen Zeitraum von drei Monaten ausgewertet; insgesamt waren vier Treffer vorhanden. Analysiert wurde das jüngste Ereignis. Es wurde keine neue Änderung in Active Directory durchgeführt.

| Prüfpunkt | Feststellung |
|---|---|
| Zeitpunkt | 17.09.2026, 09:52:38 Uhr |
| System | DC-Test - Domain Controller |
| Windows-Ereignis | Event ID 4729 - Mitglied aus Sicherheitsgruppe entfernt |
| Betroffenes Konto | SOC Rule Test |
| Privilegierte Gruppe | Domain Admins |
| Ausführendes Konto | Administrator |
| Wazuh-Erkennung | Rule ID 60159, Level 12 |
| MITRE ATT&CK | T1098.007 - Additional Local or Domain Groups |
| Taktiken | Persistence; Privilege Escalation |
| Bewertung | True Positive - Authorized Administrative Change |
| Reaktion | Keine Eindämmung erforderlich; analytisch geschlossen |

**Analystenbewertung** Die Entfernung des Kontos SOC Rule Test aus der Gruppe Domain Admins wurde tatsächlich protokolliert und von Wazuh korrekt als High-Severity-Alert erkannt. Das ausführende Konto Administrator sowie Zweck und Zeitpunkt stimmen mit dem autorisierten Labortest überein. Das Ereignis ist deshalb ein True Positive, stellt aber keinen Security Incident dar.

**Reaktion bei unautorisierter Änderung** In einer produktiven Umgebung wäre die Gruppenmitgliedschaft sofort zu prüfen und auf den freigegebenen Zustand zurückzusetzen. Zusätzlich müssten das Administrator-Konto, weitere privilegierte Änderungen und zugehörige Anmeldeereignisse untersucht werden.

**Closure Note** Authorized removal of SOC Rule Test from Domain Admins confirmed on DC-Test. Windows Event ID 4729 and Wazuh Rule 60159 (Level 12) detected the change as designed. No unauthorized privilege modification or containment requirement identified.

## 19.5 Einheitliches SOC-Triage-Runbook

Dieses Runbook standardisiert die Bearbeitung von Wazuh-Alerts im Blue-Team-Labor. Es gilt für Endpoint-, Netzwerk- und Active-Directory-Ereignisse und stellt sicher, dass jeder Alert nachvollziehbar validiert, bewertet, behandelt und geschlossen wird.

### Standardablauf

| Nr. | Phase | Prüfschritte und Dokumentation |
|---|---|---|
| **1** | **Alert aufnehmen** | Zeitpunkt, Agent/System, Rule ID, Alert-Level und Beschreibung erfassen. |
| **2** | **Technisch validieren** | Rohereignis prüfen: Event ID, Signatur, Befehl, Benutzer, Quelle, Ziel, Port und Protokoll - abhängig vom Alerttyp. |
| **3** | **Kontext prüfen** | Mit Testplan, Change-Fenster und verantwortlicher Person abgleichen. Autorisierung nicht vermuten, sondern belegen. |
| **4** | **Umfang bestimmen** | Nach zusammenhängenden Alerts, weiteren Hosts oder Konten, Wiederholungen und zeitlicher Nähe suchen. |
| **5** | **Risiko bewerten** | MITRE-Technik und -Taktik, Kritikalität des Assets, Wazuh-Level und mögliche Auswirkungen bewerten. |
| **6** | **Klassifizieren** | True Positive - Authorized Activity, True Positive - Security Incident, False Positive oder Inconclusive / Escalation Required. |
| **7** | **Reagieren** | Bei autorisierter Aktivität dokumentieren und schließen. Bei einem Incident Beweise sichern, eindämmen, zuständige Stelle informieren und weitere Analyse einleiten. |
| **8** | **Abschließen** | Ergebnis, Begründung, Maßnahmen, Analyst, Zeitstempel und Closure Note vollständig dokumentieren. |

### Eskalationskriterien

- Alert-Level 12 oder höher oder ein besonders kritisches System ist betroffen.
- Privilegierte Konten oder Gruppen wie Domain Admins wurden verändert.
- Ein codierter oder anderweitig auffälliger Befehl ist nicht eindeutig autorisiert.
- Die Quelle ist unbekannt, die Aktivität wiederholt sich oder betrifft mehrere Systeme.
- Es gibt Hinweise auf erfolgreiche Ausführung, Persistenz, Rechteausweitung oder Datenabfluss.
- Die Autorisierung kann nicht verlässlich nachgewiesen werden.

### Anwendung auf die dokumentierten Use Cases

| Use Case | Erkennung | Klassifizierung | Entscheidung |
|---|---|---|---|
| **PowerShell** | Rule 92057 / Level 12 | True Positive - Authorized Activity | Keine Eindämmung; dokumentiert und geschlossen |
| **Suricata-Portscan** | Rule 100121 / Level 8 | True Positive - Authorized Activity | Keine Eindämmung; dokumentiert und geschlossen |
| **Domain Admins** | Rule 60159 / Level 12 | True Positive - Authorized Activity | Keine Eindämmung; dokumentiert und geschlossen |

**Reaktionsleitlinie nach Alerttyp**

| Bereich | Primäre Prüfung | Mögliche Reaktion bei unautorisierter Aktivität |
|---|---|---|
| **Endpoint** | Prozessbaum, Benutzer, Command Line, Sysmon Event ID | Host isolieren, Prozess untersuchen, Beweise sichern und Konto prüfen. |
| **Netzwerk** | Quelle, Ziel, Port, Protokoll, Häufigkeit und weitere Ziele | Quelle blockieren oder segmentieren, betroffene Ziele prüfen und Verkehr sichern. |
| **Active Directory** | Event ID, ausführendes Konto, Zielkonto und Gruppenänderung | Privilegierte Änderung zurücksetzen, Konto sperren oder zurücksetzen und weitere AD-Änderungen prüfen. |

**Wiederverwendbare Closure Note**

**Vorlage** Alert [Rule ID] auf [System] wurde anhand von [Ereignis/Beweis] validiert. Die Aktivität wurde als [Klassifizierung] bewertet, weil [Begründung]. Durchgeführte Maßnahmen: [Maßnahmen oder keine Eindämmung erforderlich]. Der Fall wurde am [Zeitpunkt] durch [Analyst] geschlossen beziehungsweise an [Stelle] eskaliert.

**Ergebnis** Das Runbook verbindet die drei erfolgreich analysierten Use Cases mit einem einheitlichen, wiederverwendbaren SOC-Prozess. Damit sind Triage-Entscheidungen im Projekt konsistent, begründbar und für ein Portfolio nachvollziehbar dokumentiert.

## 19.6 Incident-Response-Ablauf am Beispiel PowerShell

Der folgende Ablauf überträgt das einheitliche Triage-Runbook auf den bereits validierten PowerShell-Alert. Das Ereignis entstand durch einen autorisierten Labortest und war kein realer Security Incident. Die Incident-Response-Maßnahmen werden deshalb als kontrolliertes Tabletop-Szenario dokumentiert; es wurden keine produktiven Konten gesperrt und kein System isoliert.

### Ausgangslage

| Prüfpunkt | Feststellung |
|---|---|
| **Betroffenes System** | ham1 - Windows 11 Endpoint |
| **Benutzer** | TEST\Administrator |
| **Erkennung** | Wazuh Rule 92057, Level 12; Sysmon Event ID 1 |
| **Beobachteter Befehl** | PowerShell mit EncodedCommand; decodierter Inhalt: Write-Output "SOC-Test" |
| **MITRE ATT&CK** | T1059.001 - PowerShell; Taktik: Execution |
| **Triage-Ergebnis** | True Positive - Authorized Activity |
| **Incident-Status** | Kein realer Incident; Tabletop-Response dokumentiert |

### Incident-Response-Phasen

| Phase | Vorgehen im Szenario | Ergebnis / Nachweis |
|---|---|---|
| **1. Preparation** | Wazuh-Agent, Sysmon-Prozessüberwachung und die integrierte Erkennungsregel sind aktiviert. Zuständigkeit, Evidenzquellen und Triage-Runbook sind definiert. | Endpoint-Telemetrie und Bearbeitungsprozess vorhanden. |
| **2. Detection & Analysis** | Alert, Sysmon Event ID 1, Benutzer und Command Line prüfen. EncodedCommand decodieren, mit dem Testplan abgleichen und nach Folgeereignissen suchen. | Autorisierter Test bestätigt; keine Hinweise auf Folgeaktivität. |
| **3. Containment** | Bei unautorisierter Aktivität: Endpoint logisch isolieren, verdächtigen Prozess stoppen und Kontoaktivität begrenzen. Im bestätigten Labortest keine Eindämmung ausführen. | Containment nicht erforderlich; Entscheidung dokumentiert. |
| **4. Eradication** | Bei bestätigtem Angriff: schädliche Skripte, Tasks oder Persistenz entfernen, Ursache identifizieren und kompromittierte Zugangsdaten zurücksetzen. | Keine Schadartefakte oder Persistenz festgestellt. |
| **5. Recovery** | Systemstatus, Wazuh-Agent und Sysmon-Telemetrie prüfen. Bei realer Isolation den Endpoint erst nach Validierung wieder freigeben und verstärkt überwachen. | System blieb betriebsbereit; Telemetrie funktionierte. |

| | | |
|---|---|---|
| **6. Lessons Learned** | Erkennung, Alert-Kontext und Reaktionsentscheidung bewerten. Autorisierte Tests künftig eindeutig kennzeichnen und Closure Note standardisiert verwenden. | Detection funktioniert; Dokumentation und Runbook wiederverwendbar. |

### Entscheidungs- und Eskalationslogik

| Bedingung | Entscheidung | Maßnahme |
|---|---|---|
| Test eindeutig autorisiert; keine Folgeaktivität | **Authorized True Positive** | Dokumentieren und analytisch schließen |
| Autorisierung unklar oder Befehl unbekannt | **Inconclusive** | An Verantwortliche eskalieren; Evidenz sichern |
| Unbekannte Quelle, schädlicher Inhalt oder Folgeaktivität | **Security Incident** | Endpoint isolieren; Konto, Prozessbaum und weitere Hosts untersuchen |
| Erkennung technisch falsch oder erwartetes Normalverhalten | **False Positive** | Regelkontext prüfen und Tuning nur nach Beleg durchführen |

### Zu sichernde Evidenz

- Wazuh-Alert mit Rule ID, Level, Agent, Zeitstempel und vollständigem Event-Kontext.
- Sysmon Event ID 1 mit Prozess, Parent Process, Benutzer und vollständiger Command Line.
- Decodierter PowerShell-Inhalt und nachvollziehbarer Abgleich mit dem autorisierten Testplan.
- Verwandte Alerts und Prozessereignisse im relevanten Zeitfenster.
- Analystenentscheidung, Maßnahmen, verantwortliche Person und Abschlusszeitpunkt.

### Kommunikation und Abschluss

**Kommunikation** Bei einem echten Incident werden Incident Owner und Systemverantwortliche informiert. Status, Risiko, betroffene Assets und bereits ausgeführte Maßnahmen werden knapp und überprüfbar festgehalten.

**Closure Note** Wazuh Rule 92057 (Level 12) detected an encoded PowerShell command on ham1 through Sysmon Event ID 1. The command decoded to Write-Output "SOC-Test" and matched the authorized lab test. No malicious follow-up activity, persistence or containment requirement was identified. The alert was classified as True Positive - Authorized Activity and closed analytically.

**Ergebnis** Der Use Case zeigt nicht nur die technische Erkennung, sondern auch einen vollständigen und nachvollziehbaren Incident-Response-Entscheidungsweg. Reale und nur simulierte Maßnahmen sind klar voneinander getrennt; dadurch bleibt die Projektdokumentation fachlich korrekt.

# 20. MITRE ATT&CK und Ereignistypen

Diese Referenz ordnet die im Projekt verwendeten Begriffe ein. Sie beschreibt Kategorien und Kennungen; sie ist keine zusätzliche Testanleitung.

## MITRE-ATT&CK-Struktur

| Begriff | Bedeutung |
|---|---|
| Tactic | Übergeordnetes Ziel eines Angreifers, z. B. Execution, Discovery, Persistence oder Privilege Escalation. |
| Technique | Beschreibt, wie ein Ziel erreicht werden kann; Kennung im Format Txxxx, z. B. T1046. |
| Sub-technique | Präzisere Variante einer Technique; Kennung im Format Txxxx.xxx, z. B. T1059.001. |
| Procedure | Konkrete beobachtete Umsetzung einer Technique durch eine Gruppe, Software oder Aktivität. |
| Mapping | Analytische Zuordnung eines beobachteten Verhaltens. Sie ersetzt nicht die technische Validierung des Events. |

## Ereignis- und Kennungstypen

| Typ | Beispiel im Projekt | Bedeutung |
|---|---|---|
| Windows Event ID | 4625, 4728, 4729 | Von Windows protokollierter Ereignistyp im jeweiligen Event Log. |
| Sysmon Event ID | 1 – Process Create | Von Sysmon erzeugte Endpoint-Telemetrie; Event ID 1 beschreibt einen Prozessstart. |
| PowerShell Event ID | 4104 – Script Block Logging | PowerShell-Protokollereignis mit erfasstem Script-Block-Inhalt, sofern Logging aktiviert ist. |
| Suricata SID | 1000001, 1000002 | Eindeutige Signatur-ID einer Suricata-Regel. Sie ist keine Wazuh Rule ID. |
| Wazuh Rule ID | 92057, 60159, 100121 | Kennung der Wazuh-Regel, die ein decodiertes Ereignis bewertet und einen Alert erzeugen kann. |
| Alert Level / Severity | Wazuh Level 8/12; Suricata Severity 2 | Produktspezifische Priorisierung. Werte verschiedener Produkte sind nicht direkt gleichzusetzen. |
| Raw Event / Alert | Windows/Sysmon/eve.json → Wazuh Alert | Das Rohereignis ist Telemetrie; ein Alert entsteht erst durch eine passende Detection-Regel. |

**Projektzuordnung: T1059.001 = PowerShell (Execution); T1046 = Network Service Discovery (Discovery); T1098.007 = Additional Local or Domain Groups (Persistence, Privilege Escalation).**